

Web Development

CSE326 · Semester 1 & 2 Study Guide

HTML5 ▪ CSS3 ▪ Modern JavaScript (ES6+) ▪ DOM Engine ▪ Regex Forms

Subject Features:

- Complete authoritative coverage of all 6 Units of the LPU CSE326 syllabus.
- Semantic HTML5 architecture, responsive CSS3 Flexbox/Grid, and modern UI design.
- Deep JavaScript fundamentals: closures, prototypes, event delegation, and asynchronous fetch.
- Form validation matrices, regular expressions, and full-length university examination papers.

Editorial Team / Publisher:

LPU Engineering Open Publishing Project

<https://github.com/LPU-Engineering-Open>

CSE326: Web Development — Study & Practice Guide

An Open-Source Educational Publication

Published by: LPU Engineering Open Initiative

Release Edition: 2026 Edition (Phase 1 Base Architecture)

Primary Source Repository: <https://github.com/LPU-Engineering-Open>

Licensing:

© 2026 The Contributors of LPU Engineering Open.

This manuscript, diagrams, and educational text are distributed under the terms of the **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** License. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

The accompanying web examples, JavaScript implementations, and style macros are distributed under the terms of the **MIT License**.

Suggested Citation:

LPU Engineering Open Contributors, *CSE326: Web Development — Study & Practice Guide*, 2026 ed. Open Educational Resources.

Important Disclaimer & Institutional Non-Affiliation

1. Independent Student-Created Resource

This book is an independent, non-profit, student-led open educational resource designed to aid students in mastering web technologies and preparing for engineering examinations.

2. Non-Affiliation Notice

LPU Engineering Open is NOT affiliated with, sponsored by, authorized by, or endorsed by Lovely Professional University (LPU). Lovely Professional University is not responsible for the contents, factual accuracy, exam representations, or editorial choices in this book.

3. Trademark & Fair Use

Any references to university curriculum structures, course codes (e.g., CSE326), subject syllabi, or exam formats are made strictly under educational **fair use** principles to provide appropriate context and academic alignment for students studying these topics.

4. Accuracy & Standards Compliance

While every effort has been made by student and educator contributors to verify standard compliance across modern browsers (W3C HTML5, WHATWG, ECMAScript), the publishers provide this material on an “as-is” basis. Readers are encouraged to report any typographical or code issues on the project’s repository.

Course Overview & Pedagogical Roadmap

Welcome to **CSE326: Web Development**, the foundational internet technologies course designed for undergraduate engineering students. This textbook synthesizes the complete 6-unit academic curriculum into a rigorous, example-driven, and examination-oriented master study guide.

Pedagogical Learning Progression

Every core web development concept throughout this book is taught using a 12-step engineering framework:

1. **Concept Definition:** What is the tag, property, function, or DOM method?
2. **Engineering Rationale:** Why is it used in web standards?
3. **Syntax Specification:** Formal syntax and attribute/parameter reference.
4. **Concrete Code Example:** Runnable, semantic HTML/CSS/JavaScript.
5. **Rendered Browser Output:** Exact terminal/browser rendering behavior.
6. **Line-by-Line Breakdown:** Granular analysis of every statement.
7. **Browser & DOM Mental Model:** Visual tree structures and execution cycles.
8. **Practical Application:** Real-world engineering implementations.
9. **Common Student Mistakes:** Traps that cause silent rendering bugs or runtime exceptions.
10. **High-Yield Exam Traps:** University examination trick questions.
11. **Graded Practice Problems:** Multi-tier problem sets (Levels 1 to 4).
12. **Complete Worked Solutions:** Full scoring solutions with marking schemes.

Contents

Course Overview & Pedagogical Roadmap	iii
1 Fundamentals of HTML (Structure, Elements & Text Formatting)	1
1.1 Unit Overview & Learning Objectives	1
1.2 Introduction to Web Technologies and HTML	1
1.2.1 The Client-Server Architecture of the Web	1
1.2.2 Defining HTML (HyperText Markup Language)	2
1.3 Structure of an HTML Document & The HTML5 Boilerplate	2
1.3.1 Standard HTML5 Boilerplate	2
1.3.2 Anatomical Breakdown of the Structural Layers	3
1.4 Root and Metadata Elements	3
1.5 Basic Structural Tags: Headings, Paragraphs, Breaks & Rules	4
1.5.1 Heading Hierarchy (<h1> through <h6>)	4
1.5.2 Paragraphs (<p>) and Whitespace Collapsing	4
1.5.3 Line Breaks () and Thematic Horizontal Rules (<hr>)	4
1.6 Text Formatting: Semantic vs. Physical Tags	5
1.7 Quotations and Comments	6
1.7.1 Block Quotations vs. Inline Quotations	6
1.7.2 HTML Comments	6
1.8 Types of HTML Tags: Paired vs. Unpaired (Void) Tags	7
1.9 Block-Level vs. Inline Elements	7
1.10 Exam Tips & Common Student Pitfalls	8
1.11 Output Prediction & Debugging Challenges	8
1.12 Graded Programming Practice Problems & Solutions	9
1.12.1 Complete Step-by-Step Worked Solutions	9
1.13 Multiple Choice Questions (MCQs)	10
1.14 Self-Assessment Unit Test	11
1.14.1 Unit Test Complete Solutions & Rubric	12
2 Expanding HTML Knowledge (Lists, Links, Media, Tables & Forms)	14
2.1 Unit Overview & Learning Objectives	14
2.2 Working with Lists (Unordered, Ordered & Definition)	14
2.2.1 1. Unordered Lists ()	14
2.2.2 2. Ordered Lists ()	15
2.2.3 3. Definition Lists (<dl>)	15
2.3 Hyperlinks and Anchor Elements (<a>)	16
2.3.1 Anchor Tag Attributes	16
2.4 Inserting Images and Favicons	16
2.4.1 The Image Tag ()	17
2.4.2 Configuring a Favicon	17
2.5 Embedding Native HTML5 Multimedia (Audio & Video)	17
2.5.1 Audio Player (<audio>)	17
2.5.2 Video Player (<video>)	18

2.6	Working with Tables (Structure & Cell Merging)	18
2.6.1	Core Table Elements	18
2.6.2	Merging Cells: <code>colspan</code> and <code>rowspan</code>	18
2.7	Working with Forms (Controls, Methods & Data Submission)	19
2.7.1	The <code><form></code> Element Attributes	19
2.7.2	Comprehensive Form Controls	19
2.8	Exam Tips & Common Student Pitfalls	21
2.9	Output Prediction & Debugging Challenges	21
2.10	Graded Programming Practice Problems & Solutions	22
2.10.1	Complete Step-by-Step Worked Solutions	23
2.11	Multiple Choice Questions (MCQs)	23
2.12	Self-Assessment Unit Test	24
2.12.1	Unit Test Complete Solutions & Rubric	25
3	Introduction to Cascading Style Sheets (CSS)	28
3.1	Unit Overview & Learning Objectives	28
3.2	Introduction to CSS & Separation of Concerns	28
3.2.1	What is CSS?	28
3.2.2	Why Use CSS?	28
3.3	CSS Insertion Methods (Types of CSS)	29
3.3.1	1. Inline CSS	29
3.3.2	2. Internal CSS	29
3.3.3	3. External CSS (Best Practice)	30
3.4	CSS Selectors (Type, ID & Class)	30
3.4.1	1. Type (Element) Selector	30
3.4.2	2. ID Selector (<code>#</code>)	30
3.4.3	3. Class Selector (<code>.</code>)	30
3.5	Typography & Text Controlling Properties	31
3.6	The CSS Box Model	31
3.6.1	Box Model Layers (Inside Out)	31
3.6.2	Box Model Shorthand Notation	33
3.7	Working with Background Images	33
3.8	Applying CSS on Tables and Forms	34
3.8.1	Professional Table Styling	34
3.8.2	Professional Form Styling	34
3.9	Exam Tips & Common Student Pitfalls	35
3.10	Output Prediction & Debugging Challenges	36
3.11	Graded Programming Practice Problems & Solutions	36
3.11.1	Complete Step-by-Step Worked Solutions	37
3.12	Multiple Choice Questions (MCQs)	38
3.13	Self-Assessment Unit Test	39
3.13.1	Unit Test Complete Solutions & Rubric	40
4	JavaScript Application Development (Syntax, Variables, Logic & Objects)	42
4.1	Unit Overview & Learning Objectives	42
4.2	Incorporating JavaScript in HTML	42
4.3	Variables: <code>var</code> , <code>let</code> , and <code>const</code>	43
4.4	Operators & Type Coercion	44
4.4.1	Arithmetic and Assignment Operators	44
4.4.2	Loose Equality (<code>==</code>) vs. Strict Equality (<code>===</code>)	44
4.4.3	Logical Operators and Short-Circuit Evaluation	44
4.5	Control Flow & Looping Statements	45

4.5.1	The <code>if...else</code> and <code>switch</code> Statements	45
4.5.2	Iterative Loops: <code>for</code> , <code>while</code> , and <code>do...while</code>	45
4.6	Browser Popup Dialogs (<code>alert</code> , <code>confirm</code> , <code>prompt</code>)	46
4.7	Working with JavaScript Objects	46
4.7.1	Object Literals, Properties & Methods	46
4.8	Exam Tips & Common Student Pitfalls	47
4.9	Output Prediction & Debugging Challenges	48
4.10	Graded Programming Practice Problems & Solutions	49
4.10.1	Complete Step-by-Step Worked Solutions	49
4.11	Multiple Choice Questions (MCQs)	50
4.12	Self-Assessment Unit Test	51
4.12.1	Unit Test Complete Solutions & Rubric	52
5	JavaScript Functions, Events and Validation	54
5.1	Unit Overview & Learning Objectives	54
5.2	Working with Functions	54
5.2.1	Syntax and Declaration	54
5.2.2	Pass-by-Value vs. Pass-by-Reference	55
5.2.3	The <code>arguments</code> Object	55
5.3	Working with JavaScript Events	56
5.3.1	The Three Event Handling Mechanisms	56
5.3.2	Catalog of Core Web Events	56
5.4	Client-Side JavaScript Form Validation	57
5.4.1	The Validation Strategy	57
5.4.2	Essential Validation Techniques	57
5.5	Exam Tips & Common Student Pitfalls	59
5.6	Output Prediction & Debugging Challenges	59
5.7	Graded Programming Practice Problems & Solutions	60
5.7.1	Complete Step-by-Step Worked Solutions	60
5.8	Multiple Choice Questions (MCQs)	61
5.9	Self-Assessment Unit Test	62
5.9.1	Unit Test Complete Solutions & Rubric	63
6	JavaScript Document Object Model (DOM)	65
6.1	Unit Overview & Learning Objectives	65
6.2	DOM Architecture & The Document Tree	65
6.3	The <code>document</code> Root Object	66
6.4	DOM Element Selection Methods	67
6.5	DOM Elements: Creation, Deletion, and Tree Traversal	67
6.5.1	Creating and Appending Nodes	67
6.5.2	DOM Tree Traversal Properties	67
6.6	DOM HTML: Manipulating Content and Attributes	69
6.6.1	Content Manipulation: <code>innerHTML</code> vs. <code>textContent</code>	69
6.7	DOM CSS: Dynamic Styling & The <code>classList</code> API	69
6.7.1	1. The <code>style</code> Property (Inline Styling)	69
6.7.2	2. The <code>classList</code> API (Best Practice)	70
6.8	DOM Events, The Event Object & Event Propagation	70
6.8.1	The <code>Event</code> Object	70
6.8.2	Event Propagation: Bubbling vs. Capturing	70
6.9	Exam Tips & Common Student Pitfalls	71
6.10	Output Prediction & Debugging Challenges	71
6.11	Graded Programming Practice Problems & Solutions	72

6.11.1	Complete Step-by-Step Worked Solutions	72
6.12	Multiple Choice Questions (MCQs)	73
6.13	Self-Assessment Unit Test	74
6.13.1	Unit Test Complete Solutions & Rubric	75
7	Master Web Cheatsheet & Top 50 Technical Viva-Voce Questions	78
7.1	Master HTML5 Syntax & Semantic Tag Matrix	78
7.2	HTML5 Form Controls & Multimedia Cheatsheet	78
7.3	CSS Selectors, Specificity & Box Model Matrix	78
7.4	JavaScript Operators, DOM Methods & Events Synthesis	78
7.5	Top 50 Technical Viva-Voce & Examination Questions	78
8	Comprehensive Practical Web Projects	85
8.1	Project 1: Developer Portfolio Profile Page	85
8.2	Project 2: Academic Timetable & Course Portal	87
8.3	Project 3: Interactive Semester CGPA Calculator	88
8.4	Project 4: Comprehensive Registration Form Validator	90
8.5	Project 5: Interactive Dynamic To-Do List Application	92
8.6	Project 6: Real-time Filterable Student Directory	93
8.7	Project 7: Interactive Accordion FAQ Component	94
8.8	Project 8: Interactive Lightbox Image Gallery	96
9	Full-Length University Mock Examinations	98
9.1	University Mock Examination 1	98
9.1.1	Complete Worked Solutions for Mock Examination 1	99
9.2	University Mock Examination 2	100
9.2.1	Complete Worked Solutions for Mock Examination 2	101
9.3	University Mock Examination 3	103
9.3.1	Complete Worked Solutions for Mock Examination 3	104
9.4	University Mock Examination 4	105
9.4.1	Complete Worked Solutions for Mock Examination 4	106
9.5	University Mock Examination 5	107
9.5.1	Complete Worked Solutions for Mock Examination 5	109
9.6	University Mock Examination 6	110
9.6.1	Complete Worked Solutions for Mock Examination 6	111

Fundamentals of HTML (Structure, Elements & Text Formatting)

1.1 Unit Overview & Learning Objectives

Unit 1 Academic Scope

LPU CSE326

HyperText Markup Language (HTML) is the universal structural foundation of the World Wide Web. This unit introduces web architecture, the Client-Server model, HTML5 document boilerplate structure, root and metadata elements, basic content tags, semantic text formatting, quotations, comments, paired versus unpaired (void) tags, and the fundamental behavioral dichotomy between block-level and inline elements.

- **Web Architecture Foundations:** Understand client-side browser rendering, web server hosting, and HTTP/HTTPS client-server request-response cycles.
- **HTML5 Document Anatomy:** Master the standard HTML5 boilerplate, distinguishing metadata declarations in `<head>` from visible user content in `<body>`.
- **Metadata Management:** Configure character encodings (UTF-8), mobile viewport scalings, external stylesheet links, and document titles for SEO.
- **Structural & Content Markup:** Construct hierarchical documents using semantic headings (`<h1>`–`<h6>`), paragraph flows (`<p>`), line breaks (`<br>`), and thematic dividers (`<hr>`).
- **Semantic Formatting:** Contrast stylistic formatting with semantic tags (`<strong>`, `<em>`, `<mark>`, `<small>`, `<abbr>`, `<cite>`, `<dfn>`).
- **Element Display Categories:** Differentiate the layout behaviors, margin models, and containment rules of block-level versus inline elements.

1.2 Introduction to Web Technologies and HTML

1.2.1 The Client-Server Architecture of the Web

The World Wide Web operates upon a distributed **Client-Server Architecture**:

1. **The Client (Browser):** A client software program (e.g., Google Chrome, Mozilla Firefox, Apple Safari) that initiates requests for web resources. The browser's layout engine parses HTML, constructs the Document Object Model (DOM), computes CSS styles, executes JavaScript, and renders visual pixels onto the screen.

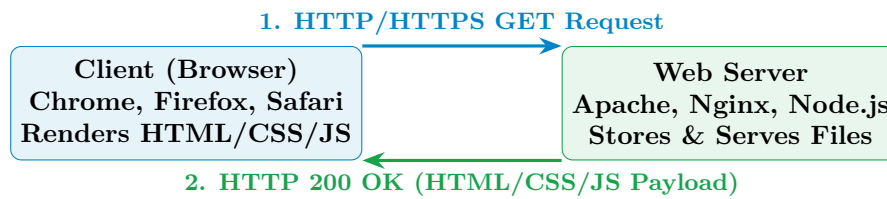


Figure 1.1: The Client-Server HTTP Request-Response Interaction.

2. **The Web Server:** A remote computer running web server software (e.g., Nginx, Apache) listening on network ports (Port 80 for HTTP, Port 443 for HTTPS) that locates requested files on disk and transmits them back to the client.
3. **HTTP / HTTPS: HyperText Transfer Protocol** (and its TLS-encrypted variant, **HTTPS**) is the application-layer communication protocol governing the transmission of web resources across TCP/IP networks.

1.2.2 Defining HTML (HyperText Markup Language)

- **HyperText:** Text containing embedded interactive links (**hyperlinks**) that enable non-linear navigation between documents across the web.
- **Markup Language:** A system of annotating text using syntactic **tags** (e.g., `<p>`, `<h1>`) to describe the structural purpose, formatting, and semantic role of content to the rendering engine.
- **HTML5 Standards:** Standardized by the W3C and WHATWG, HTML5 introduced native multimedia elements (`<audio>`, `<video>`), semantic layout landmarks (`<header>`, `<nav>`, `<article>`, `<section>`, `<footer>`), dynamic vector canvases (`<canvas>`), and client-side storage APIs.

1.3 Structure of an HTML Document & The HTML5 Boilerplate

Every valid HTML5 document follows a strict tree hierarchy representing the Document Object Model (DOM).

1.3.1 Standard HTML5 Boilerplate

HTML: Standard HTML5 Document Template

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
6     initial-scale=1.0">
7   <title>CSE326: University Portal</title>
8 </head>
9 <body>
10   <!-- Visible page content begins here -->
11   <h1>Welcome to Web Development</h1>
12   <p>This is a foundational HTML5 engineering document.</p>
13 </body>
14 </html>
```

Browser Viewport Window

Heading 1: Welcome to Web Development Paragraph: This is a foundational HTML5 engineering document.

1.3.2 Anatomical Breakdown of the Structural Layers

1. **<!DOCTYPE html>**: This is a document type declaration (DTD), **not an HTML tag**. It instructs modern web browsers to parse the page in standard compliance mode rather than legacy "quirks mode".
2. **<html lang="en">**: The top-level **root element** enclosing all other tags. The **lang** attribute specifies the primary natural language (e.g., **"en"** for English), aiding screen readers and search engines.
3. **<head>**: The non-visible **metadata container** holding document configuration, character sets, viewport scaling rules, external CSS stylesheets, and page titles.
4. **<body>**: The **visible content container** containing all markup rendered within the browser's viewport (headings, paragraphs, tables, images, scripts).

1.4 Root and Metadata Elements

Metadata elements reside strictly inside the **<head>** section to provide machine-readable metadata about the HTML document:

Metadata Tag	Engineering Purpose & Functionality	Example Code
<code><title></code>	Defines text displayed in browser tab and search engine results	<code><title>Home</title></code>
<code><meta charset></code>	Specifies character encoding supporting global multilingual characters	<code><meta charset="UTF-8"></code>
<code><meta viewport></code>	Configures responsive mobile scaling to match device screen width	<code><meta name="viewport" ...></code>
<code><meta description></code>	Summary text indexable by search engine web crawlers	<code><meta name="description" ...></code>
<code><link></code>	Links external resources (external CSS stylesheets, favicons)	<code><link rel="stylesheet" ...></code>
<code><style></code>	Encloses internal cascading style sheet (CSS) declarations	<code><style> p { color: red; } </style></code>
<code><script></code>	Embeds or imports external JavaScript executable logic	<code><script src="app.js"></script></code>

Table 1.1: Core Metadata Elements within the HTML Head Section.

The Critical Role of the Viewport Meta Tag

The viewport tag `<meta name="viewport" content="width=device-width, initial-scale=1.0">` is essential for **Responsive Web Design (RWD)**. `width=device-width` instructs the browser to set the page width equal to the physical screen width of the device in device-independent pixels, while `initial-scale=1.0` establishes a 1:1 zoom ratio upon initial page load, preventing mobile browsers from rendering desktop-width layouts zoomed out.

1.5 Basic Structural Tags: Headings, Paragraphs, Breaks & Rules

1.5.1 Heading Hierarchy (<h1> through <h6>)

HTML provides six levels of sectional headings. Headings are **block-level elements** rendered with bold font weights and descending font sizes:

HTML: Heading Levels Demonstration

```
1 <h1>Level 1: Main Document Subject</h1>
2 <h2>Level 2: Major Section Heading</h2>
3 <h3>Level 3: Subsection Heading</h3>
4 <h4>Level 4: Sub-subsection Heading</h4>
5 <h5>Level 5: Minor Topic Title</h5>
6 <h6>Level 6: Lowest Level Heading</h6>
```

Heading Best Practices

- Use **exactly one** `<h1>` tag per webpage representing the primary page topic for search engine optimization (SEO) and accessibility.
- Do NOT skip heading levels (e.g., going directly from `<h1>` to `<h3>`) as this breaks screen-reader outline hierarchies.
- Never use heading tags purely to make text bold or large; use CSS styling for visual adjustments.

1.5.2 Paragraphs (<p>) and Whitespace Collapsing

The `<p>` tag encapsulates a block of running prose.

Browser Whitespace Collapsing

By default, web browsers automatically collapse consecutive whitespace characters (spaces, tabs, newlines) within standard HTML tags into a **single space character**. To preserve literal whitespace and line formatting exactly as written, use the preformatted text tag `<pre>...</pre>`.

1.5.3 Line Breaks (
) and Thematic Horizontal Rules (<hr>)

- **
**: Inserts an inline line break (carriage return) without starting a new paragraph block. It is a void/unpaired element.

- **<hr>**: Inserts a thematic break, visually rendered as a horizontal dividing line separating distinct content sections.

HTML: Paragraphs, Line Breaks, and Horizontal Rules

```

1 <p>
2   Lovely Professional University<br>
3   Jalandhar - Delhi G.T. Road, Phagwara<br>
4   Punjab, India - 144411
5 </p>
6 <hr>
7 <p>Department of Computer Science and Engineering.</p>

```

1.6 Text Formatting: Semantic vs. Physical Tags

Modern HTML strictly distinguishes **Semantic Tags** (which convey meaning to assistive technologies and parsers) from **Physical/Presentational Tags** (which only dictate visual styles):

Semantic Tag	Visual Default	Semantic Meaning / Accessibility Role	Contrast with Physical Tag
	Bold	Serious importance, urgency, or critical alert	 (Bold without importance)
	<i>Italic</i>	Stress emphasis that alters vocal nuance	<i> (Italic alternate voice / technical term)
<mark>	Highlighted	Text marked/highlighted for reference	Yellow highlight
<small>	Smaller font	Fine print, legal disclaimers, copyrights	Structural side comments
<abbr>	Normal text	Abbreviation/acronym; full term in title	Tooltip expansion upon hover
<cite>	<i>Italic</i>	Title of a creative work (book, paper, movie)	Citation reference
<dfn>	<i>Italic</i>	Defining instance of a technical term	Term being defined in surrounding text
	Strikethrough	Deleted / superseded content	Historical change tracking
<sub>	Subscript	Inferior subscript positioning (H_2O)	Baseline offset
<sup>	Superscript	Superior superscript positioning (X^2)	Mathematical exponent / footnote

Table 1.2: Semantic and Presentational Text Formatting Tags in HTML5.

HTML: Semantic Text Formatting in Action

```

1 <p>

```

```
2     The <dfn><abbr title="World Wide Web
    Consortium">W3C</abbr></dfn> is the
3     <strong>primary international standards organization</strong>
    for the web.
4 </p>
5 <p>
6     Einstein formulated the mass-energy equivalence: <em>E =
    mc<sup>2</sup></em>.
7 </p>
8 <p>
9     Chemical formula for glucose:
    C<sub>6</sub>H<sub>12</sub>O<sub>6</sub>.
10 </p>
11 <p>
12     Registration Fee: <del>Rs. 1000</del> <mark>Rs. 500
    (Discounted)</mark>.
13 </p>
```

1.7 Quotations and Comments

1.7.1 Block Quotations vs. Inline Quotations

1. **Block Quotations (<blockquote>):** Used for extended multi-line quotations. Browsers typically render <blockquote> as an indented block with top and bottom margins. The optional cite attribute supplies the source URL.
2. **Inline Quotations (<q>):** Used for brief, inline quotations within a paragraph. Browsers automatically wrap the enclosed text with language-appropriate quotation marks.

HTML: Quotations Demonstration

```
1 <!-- Multi-line Blockquote -->
2 <blockquote cite="https://www.w3.org">
3     The power of the Web is in its universality. Access by
    everyone regardless of disability is an essential aspect.
4 </blockquote>
5
6 <!-- Inline Quote -->
7 <p>Tim Berners-Lee stated, <q>The Web does not just connect
    machines, it connects people.</q></p>
```

1.7.2 HTML Comments

Comments are ignored by the browser's rendering engine but remain visible in page source inspection:

Syntax Specification: HTML Comment Syntax

```
<!-- This is an HTML comment that will not render on screen -->
```

1.8 Types of HTML Tags: Paired vs. Unpaired (Void) Tags

Category	Definition & Syntactic Rules	Tag Examples
Paired Tags	Require both an opening tag <code><tag></code> and a closing tag <code></tag></code> to enclose content.	<code><html></code> , <code><body></code> , <code><p></code> , <code><h1></code> , <code><div></code> , <code></code> , <code></code>
Unpaired (Void) Tags	Self-contained elements that cannot contain inner text or children ; no closing tag permitted.	<code>
</code> , <code><hr></code> , <code></code> , <code><input></code> , <code><meta></code> , <code><link></code>

Table 1.3: Paired Container Elements versus Unpaired Void Elements.

1.9 Block-Level vs. Inline Elements

Understanding element display classification is essential for mastering page layout geometry:

Architectural Feature	Block-Level Elements	Inline Elements
Line Break Behavior	Always starts on a new line	Stays inline; flows with surrounding text
Default Width	Expands to 100% width of parent container	Width wraps tightly around its content
Vertical Margins & Padding	Fully respects top/bottom margins and padding	Top/bottom margins do not alter line layout
Containment Rules	Can contain other block and inline elements	Can only contain data and other inline elements
Core Examples	<code><div></code> , <code><p></code> , <code><h1></code> – <code><h6></code> , <code></code> , <code></code> , <code></code> , <code><section></code>	<code></code> , <code><a></code> , <code></code> , <code></code> , <code></code> , <code><mark></code> , <code><abbr></code>

Table 1.4: Comprehensive Comparison of Block-Level vs. Inline Elements.

HTML: Visualizing Block vs Inline Layout Behavior

```
1  <!-- Block elements: Each renders on a distinct vertical line
   spanning full width -->
2  <div style="background-color: lightblue;">Block Container 1</div>
3  <div style="background-color: lightcoral;">Block Container 2</div>
4
5  <!-- Inline elements: Render side-by-side on the same horizontal
   line -->
6  <span style="background-color: yellow;">Inline Span 1</span>
```

```
7 <span style="background-color: lightgreen;">Inline Span 2</span>
```

1.10 Exam Tips & Common Student Pitfalls

Unit 1 University Exam Traps

- **Void Tag Closing:** Writing `<br></br>` or `<hr></hr>` is invalid HTML5 syntax. Void tags have no closing tags.
- **Block Elements inside Inline Elements:** Placing a `<div>` or `<p>` inside a `<span>` or `<strong>` violates HTML nesting rules and breaks layout trees.
- **`<strong>` vs `<b>`:** `<b>` changes only visual appearance; `<strong>` represents importance for screen readers and SEO algorithms.

1.11 Output Prediction & Debugging Challenges

Output Prediction: Whitespace Collapsing and Text Formatting

Question: Predict the exact visual layout rendered by the browser:

```
1 <p>First      Line    of  
2 Text.<br>Second Line: <strong>Bold <em>and  
   Emphasized</em></strong>.</p>
```

Answer & Layout Tracing:

- The spaces between `First`, `Line`, `of`, and `Text.` collapse into single spaces: "First Line of Text."
- The `<br>` forces a line break.
- Line 2 renders "Second Line: " followed by "Bold " in bold, and "and Emphasized" in *bold italic*.

Debugging Challenge: Fixing Structural Nesting and Void Tag Errors

Buggy Code:

```
1 <span>  
2   <h1>Chapter 1</h1>  
3   <p>Welcome to CSE326.<br></br></p>  
4 </span>
```

Diagnosis & Correction:

1. **Bug 1:** Block-level elements (`<h1>`, `<p>`) are illegally nested inside an inline `<span>`. Replace `<span>` with a block `<div>`.
2. **Bug 2:** `<br></br>` contains an illegal closing tag. Use `<br>` as a void element.

Corrected Code:

```

1 <div>
2   <h1>Chapter 1</h1>
3   <p>Welcome to CSE326.<br></p>
4 </div>

```

1.12 Graded Programming Practice Problems & Solutions

Unit 1 Practice Problems

- Level 1 (Foundation):** Write a complete HTML5 document structure containing a page title, an `<h1>` heading with your university name, and a paragraph describing your degree program.
- Level 1 (Foundation):** Construct a poem formatting snippet using paragraphs and line breaks (`<br>`) that prevents whitespace collapsing.
- Level 2 (Standard):** Create an academic profile card containing an abbreviation with expansion (`<abbr>`), highlighted GPA score (`<mark>`), and deleted previous course code (`<del>`).
- Level 2 (Standard):** Write HTML code demonstrating the chemical equation for water synthesis ($2H_2 + O_2 \rightarrow 2H_2O$) and the quadratic formula ($ax^2 + bx + c = 0$) using `<sub>` and `<sup>`.
- Level 3 (Exam Tier):** Create a structured news article layout containing a main heading, a publication date in `<small>`, an introductory paragraph, a multi-line quotation with source attribution (`<blockquote>`), and an inline quote (`<q>`).
- Level 3 (Exam Tier):** Construct an HTML snippet illustrating the layout difference between three colored block-level `<div>` containers and three colored inline `<span>` phrasing tags.
- Level 4 (Challenge):** Build a standards-compliant HTML5 technical terms glossary using `<dfn>`, `<abbr>`, `<strong>`, and comments explaining each section.

1.12.1 Complete Step-by-Step Worked Solutions

Worked Solution: Solutions to Unit 1 Practice Problems

Solution to Problem 4 (Subscript and Superscript Equations):

```

1 <p>
2   <strong>Chemical Reaction:</strong><br>
3   2H<sub>2</sub> + O<sub>2</sub> &rarr; 2H<sub>2</sub>O
4 </p>
5 <p>
6   <strong>Quadratic Polynomial Equation:</strong><br>
7   <em>f(x) = ax<sup>2</sup> + bx + c</em>
8 </p>

```

Solution to Problem 5 (Structured News Article with Quotes):

```

1 <!DOCTYPE html>

```

```
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Global Tech News</title>
6 </head>
7 <body>
8   <h1>Quantum Computing Breakthrough Announced</h1>
9   <p><small>Published: August 29, 2026 | Technology
10  Desk</small></p>
11   <p>Researchers have achieved quantum advantage in complex
12  optimization.</p>
13   <blockquote cite="https://science.org/quantum">
14     This computational leap enables simulations of molecular
15     dynamics previously considered impossible in classical computing
16     architectures.
17   </blockquote>
18   <p>The lead scientist noted, <q>We are entering a new era of
19  physical computing.</q></p>
20 </body>
21 </html>
```

1.13 Multiple Choice Questions (MCQs)

1. What is the correct HTML5 document type declaration?

- (a) `<!DOCTYPE html>` (b) `<DOCTYPE html5>` (c) `<html doctype="5">` (d) `<?xml html>`

Answer: (a) — *<!DOCTYPE html> is the standard case-insensitive HTML5 DTD.*

2. Which HTML element contains metadata not displayed directly within the browser viewport?

- (a) `<body>` (b) `<head>` (c) `<section>` (d) `<footer>`

Answer: (b) — *Metadata tags reside inside <head>.*

3. Which tag is an example of an unpaired (void) element in HTML5?

- (a) `<p>` (b) `<strong>` (c) `<hr>` (d) `<div>`

Answer: (c) — *<hr> is a void element without a closing tag.*

4. How do modern browsers handle consecutive spaces and newlines in HTML source?

- (a) Preserves all spaces (b) Collapses into a single space (c) Raises error (d) Deletes all spaces

Answer: (b) — *Browsers collapse consecutive whitespace characters into one space.*

5. Which tag defines the title of a creative work such as a book or painting?

- (a) `<cite>` (b) `<q>` (c) `<dfn>` (d) `<abbr>`

Answer: (a) — *<cite> defines the title of a creative work.*

6. What is the default display width of a block-level element?

- (a) 0% (b) Width of its text content (c) 100% of parent width (d) 50% of screen

Answer: (c) — *Block elements stretch to 100% width of their parent container.*

7. Which tag is semantically appropriate for marking text with strong importance?

- (a) `<b>` (b) `<strong>` (c) `<bold>` (d) `<heavy>`

Answer: (b) — *`<strong>` conveys semantic importance for accessibility.*

8. Which attribute in the `<abbr>` tag provides the expanded description?

- (a) `alt` (b) `src` (c) `title` (d) `name`

Answer: (c) — *The `title` attribute specifies the abbreviation tooltip.*

9. What is the correct syntax for an HTML comment?

- (a) `// Comment` (b) `/* Comment */` (c) `<!-- Comment ->` (d) `# Comment`

Answer: (c) — *HTML comments use `<!-- ... ->`.*

10. Which tag creates an inline quotation with automatic quotation punctuation?

- (a) `<blockquote>` (b) `<q>` (c) `<quote>` (d) `<cite>`

Answer: (b) — *`<q>` inserts inline quotation marks.*

1.14 Self-Assessment Unit Test

Unit 1 Examination (25 Marks — 45 Minutes)

Section A: Short Answer Concepts (3 × 2 = 6 Marks)

1. Explain the significance of the `<!DOCTYPE html>` declaration.
2. Differentiate between `<strong>` and `<b>` with respect to web accessibility.
3. State three differences between block-level and inline elements.

Section B: Code Tracing & Output Prediction (3 × 3 = 9 Marks)

4. Predict the output of this HTML snippet:

```
1 <p>Water: H<sub>2</sub>O | Energy: E = mc<sup>2</sup></p>
2
```

5. Identify the errors in the following code:

```
1 <head>
2   <title>My Portal</title>
3   <p>Welcome to the portal.</p>
4 </head>
5
```

6. What is the difference between `<blockquote>` and `<q>`?

Section C: Comprehensive Programming Problem (1 × 10 = 10 Marks)

7. Write a complete, standards-compliant HTML5 document for a University Course Information page:
 - (a) Include proper metadata tags (UTF-8, responsive viewport, page title). (3 Marks)
 - (b) Structure the document with `<h1>`, `<h2>`, paragraphs, and an address using `<br>`. (3 Marks)
 - (c) Use semantic formatting tags (`<strong>`, `<em>`, `<mark>`, `<abbr>`, `<hr>`). (4 Marks)

1.14.1 Unit Test Complete Solutions & Rubric

Worked Solution: Complete Solutions for Unit 1 Test

Section A Solutions:

1. **DOCTYPE Declaration** [2 Marks]: Instructs browsers to render pages in HTML5 standard mode rather than quirks mode.
2. **Strong vs. Bold** [2 Marks]: `<b>` provides visual bolding without semantic weight; `<strong>` adds semantic importance, causing screen readers to emphasize the text vocally.
3. **Block vs. Inline** [2 Marks]: Block elements start on a new line, expand to 100% width, and support vertical margins. Inline elements flow horizontally, take only content width, and do not disrupt line height with top/bottom margins.

Section B Solutions:

4. **Sub/Sup Output** [3 Marks]: Renders "Water: H²" with subscript "2" and "O", followed by " | Energy: E = mc²".
5. **Head Element Error** [3 Marks]: `<p>` is a visible content element and cannot reside inside `<head>`. It must be moved inside the `<body>` container.
6. **Quote Tags** [3 Marks]: `<blockquote>` is a block-level container for long indented excerpts; `<q>` is an inline phrasing element with automatic quotation marks.

Section C Solution:

7. **Complete HTML5 Document** [10 Marks]:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width,
      initial-scale=1.0">
6     <title>CSE326 - Course Information</title>
7 </head>
8 <body>
9     <h1>Department of Computer Science & IT</h1>
10    <h2>CSE326: Web Development</h2>
11    <hr>
12    <p>

```

```
13      Welcome to <strong>CSE326</strong>. This course covers
14      <abbr title="HyperText Markup Language">HTML5</abbr>,
15      <abbr title="Cascading Style Sheets">CSS3</abbr>, and
16      <em>JavaScript</em>.
17      </p>
18      <p>
19          <mark>Prerequisite:</mark> Basic programming
20      fundamentals.
21      </p>
22      <p>
23          <strong>Academic Office:</strong><br>
24          Room 34-201, Innovation Block<br>
25          Lovely Professional University
26      </p>
27 </body>
</html>
```

Marking Scheme: DOCTYPE & metadata: 3M; Hierarchy & paragraphs: 3M; Semantic tags: 4M.

Expanding HTML Knowledge (Lists, Links, Media, Tables & Forms)

2.1 Unit Overview & Learning Objectives

Unit 2 Academic Scope

LPU CSE326

Static text alone cannot build interactive web experiences. This unit covers complex HTML data structures including unordered, ordered, and definition lists, hyperlinks, images, native HTML5 audio and video embedding, favicons, advanced tabular data presentation with cell merging (`colspan` and `rowspan`), and comprehensive web form architectures for client data collection.

- **List Structuring:** Author unordered (`<ul>`), ordered (`<ol>`), and definition glossary lists (`<dl>`) with custom numbering types and start offsets.
- **Navigation & Hyperlinks:** Build local, absolute, and bookmark links using the anchor tag (`<a>`) and configure tab targets.
- **Multimedia Embedding:** Embed raster images (`<img>`), native audio streams (`<audio>`), and video players (`<video>`) with controls and poster thumbnails.
- **Tabular Data Presentation:** Design complex tables (`<table>`, `<tr>`, `<th>`, `<td>`) utilizing horizontal (`colspan`) and vertical (`rowspan`) cell spanning.
- **Form Controls & Data Collection:** Build interactive forms with text inputs, password masking, radio buttons, checkboxes, dropdown select menus, file uploads, and submit/reset actions.

2.2 Working with Lists (Unordered, Ordered & Definition)

HTML provides three primary list architectures to group related items:

2.2.1 1. Unordered Lists (`<ul>`)

Unordered lists represent collections where sequence is non-ordinal, displayed with bullet markers:

Syntax Specification: Unordered List Syntax

```
<ul type="disc|circle|square"> <li>List item text</li> </ul>
```

HTML: Unordered Lists in HTML

```

1 <ul>
2   <li>HTML5 (Structural Layer)</li>
3   <li>CSS3 (Presentation Layer)</li>
4   <li>JavaScript (Behavioral Layer)</li>
5 </ul>

```

2.2.2 2. Ordered Lists ()

Ordered lists represent sequences where item order matters, displayed with numerical or alphabetical indices:

type Value	Numbering Style	Example Output	Use Case
'1' (Default)	Decimal Numbers	1, 2, 3, ...	Step-by-step algorithms, tutorials
'A'	Uppercase Letters	A, B, C, ...	Major outline sections
'a'	Lowercase Letters	a, b, c, ...	Multiple-choice options
'I'	Uppercase Roman	I, II, III, ...	Classical book chapters
'i'	Lowercase Roman	i, ii, iii, ...	Preface, appendix listings

Table 2.1: Ordered List type Attribute Values.

HTML: Ordered List with Custom Type and Start Offset

```

1 <!-- Starts numbering from Roman numeral III (value 3) -->
2 <ol type="I" start="3">
3   <li>System Architecture Analysis</li>
4   <li>Interface Wireframing</li>
5   <li>Implementation and Testing</li>
6 </ol>

```

Browser Rendered Output / Console Display

```

III. System Architecture Analysis IV. Interface Wireframing V. Implementation and
Testing

```

2.2.3 3. Definition Lists (<dl>)

Definition lists structure glossaries or key-value terms:

- **<dl>**: Encloses the entire description list.
- **<dt>**: Defines the ****definition term****.
- **<dd>**: Defines the ****definition description**** (indented by default).

HTML: Definition List Glossary

```

1 <dl>
2   <dt>HTTP</dt>
3   <dd>HyperText Transfer Protocol - Application protocol for
4     data communication.</dd>
5   <dt>DOM</dt>

```

```

5      <dd>Document Object Model - Object representation of the HTML
      document tree.</dd>
6 </dl>

```

2.3 Hyperlinks and Anchor Elements (<a>)

Hyperlinks connect web documents across local directories and global networks.

2.3.1 Anchor Tag Attributes

Syntax Specification: Anchor Element

```
<a href="destination_url" target="_self|_blank" title="tooltip_text">Link Label</a>
```

Target Value	Window Behavior	Security / Usability Note
<code>_self</code> (Default)	Opens the destination in the **same browser tab/window**	Standard intra-site navigation
<code>_blank</code>	Opens the destination in a **new browser tab/window**	External links; recommend rel="noopener"
<code>_parent</code>	Opens link in parent frame of a frameset/iframe	Nested iframe navigation
<code>_top</code>	Opens link in full body of current window	Escapes iframe containers

Table 2.2: Anchor Element `target` Attribute Options.

HTML: Various Hyperlink Types

```

1  <!-- Absolute External Link -->
2  <a href="https://www.lpu.in" target="_blank" title="Visit
   University Portal">LPU Official Website</a>
3
4  <!-- Relative Internal Link -->
5  <a href="about.html">About Department</a>
6
7  <!-- Bookmark / Fragment Identifier Link -->
8  <a href="#syllabus-section">Jump to Syllabus</a>
9
10 <!-- Email Link -->
11 <a href="mailto:support@lpuverto.xyz">Contact Support Email</a>

```

2.4 Inserting Images and Favicons

2.4.1 The Image Tag ()

The tag embeds raster or vector images. It is a **void element** (no closing tag):

HTML: Inserting Images with Accessibility Attributes

```
1 
```

The Critical Importance of the alt Attribute

The **alt** (alternative text) attribute serves three critical roles:

1. **Accessibility:** Read aloud by screen readers for visually impaired users.
2. **Network Fallback:** Rendered in the viewport if the image fails to load due to broken URLs or slow connections.
3. **Search Engine Optimization (SEO):** Indexed by search engine web crawlers to categorize image content.

2.4.2 Configuring a Favicon

A favicon is a branding icon displayed in the browser tab beside the page title. It is declared inside <head> using <link>:

HTML: Favicon Declaration in Head

```
1 <head>
2   <title>Student Dashboard</title>
3   <link rel="icon" type="image/x-icon"
4     href="/assets/favicon.ico">
5 </head>
```

2.5 Embedding Native HTML5 Multimedia (Audio & Video)

HTML5 eliminated the need for third-party browser plugins (such as Adobe Flash) by introducing native multimedia tags:

2.5.1 Audio Player (<audio>)

HTML: HTML5 Audio Player with Fallback

```
1 <audio controls loop muted>
2   <source src="lecture_audio.mp3" type="audio/mpeg">
3   <source src="lecture_audio.ogg" type="audio/ogg">
4   Your browser does not support the audio element.
5 </audio>
```

2.5.2 Video Player (<video>)

HTML: HTML5 Video Player with Poster Frame

```

1 <video width="640" height="360" controls poster="thumbnail.jpg">
2   <source src="web_dev_lecture.mp4" type="video/mp4">
3   <source src="web_dev_lecture.webm" type="video/webm">
4   Your browser does not support the video tag.
5 </video>

```

Media Attribute	Type	Functionality
controls	Boolean	Displays playback UI (play/pause button, volume slider, timeline scrubber)
autoplay	Boolean	Starts playback automatically upon page load (browsers require muted)
loop	Boolean	Automatically restarts media from beginning upon reaching end
muted	Boolean	Silences audio output by default
poster	URL String	Displays preview image before video playback begins (<video> only)

Table 2.3: HTML5 Media Attributes Reference.

2.6 Working with Tables (Structure & Cell Merging)

HTML tables present two-dimensional tabular data in rows and columns:

2.6.1 Core Table Elements

- **<table>**: The outermost container for tabular data.
- **<tr>**: Defines a **table row**.
- **<th>**: Defines a **table header cell** (rendered bold and centered by default).
- **<td>**: Defines a standard **table data cell** (left-aligned by default).

2.6.2 Merging Cells: colspan and rowspan

1. **colspan="n"**: Merges a cell horizontally across *n* adjacent columns.
2. **rowspan="n"**: Merges a cell vertically across *n* adjacent rows.

HTML: Complex Table with Colspan and Rowspan

```

1 <table border="1" style="border-collapse: collapse; width: 100%;">
2   <thead>
3     <tr>
4       <th>Student Name</th>
5       <th colspan="2">Contact Information</th>
6       <th>Status</th>
7     </tr>
8   </thead>

```

```

 9      <tbody>
10      <tr>
11          <td rowspan="2">Aarav Sharma</td>
12          <td>Email: aarav@lpu.in</td>
13          <td>Phone: +91-9876543210</td>
14          <td rowspan="2">Enrolled</td>
15      </tr>
16      <tr>
17          <td colspan="2">Address: Block 34, LPU Campus,
Phagwara</td>
18      </tr>
19  </tbody>
20 </table>

```

Browser Rendered Output / Console Display

```

+-----+-----+-----+-----+ | Student Name | Contact
Information | Status | +-----+-----+-----+-----+ |
Aarav Sharma | Email: aarav@lpu.in | Phone: 9876543210 | Enrolled | |
+-----+-----+-----+ | | Address: Block 34, LPU Campus, Phagwara | |
+-----+-----+-----+-----+

```

2.7 Working with Forms (Controls, Methods & Data Submission)

Forms collect user input and transmit payloads to server endpoints for processing.

2.7.1 The <form> Element Attributes

Syntax Specification: Form Tag Structure

```

<form action="endpoint_url" method="GET|POST" enctype="multipart/form-data"> <!--
Form Controls --> </form>

```

Feature	GET Method	POST Method
Data Location	Appended to URL query string (?q=term&page=1)	Transmitted inside HTTP request body payload
Security	Insecure: Visible in URL, logs, and browser history	Secure: Concealed from URL visibility
Data Limit	Limited by URL length restrictions (≈ 2048 chars)	Virtually unlimited payload capacity
Bookmarking	URL can be bookmarked / cached	Cannot be bookmarked directly
Primary Use	Search queries, filter parameters, idempotent lookups	Passwords, registration forms, file uploads

Table 2.4: Comparison of HTTP Form Submission Methods: GET vs. POST.

2.7.2 Comprehensive Form Controls

1. Text Inputs (<input type="...">):

- `type="text"`: Single-line alphanumeric text.
 - `type="password"`: Obscures typed characters with masking dots.
 - `type="email"`: Client-side pattern format verification.
 - Attributes: `placeholder`, `name`, `required`, `value`, `maxlength`.
2. **Multi-line Text Area (`<textarea>`)**: Provides expandable multi-line text input configured via `rows` and `cols`.
 3. **Radio Buttons (`type="radio"`)**: Permits selection of **exactly one option** from a mutually exclusive group. All radio buttons in the group **must share the identical `name` attribute**.
 4. **Checkboxes (`type="checkbox"`)**: Permits independent toggling of zero or more options.
 5. **Dropdown Selection Menu (`<select>`)**: Displays compact dropdown menus populated by child `<option>` tags.
 6. **File Upload (`type="file"`)**: Enables file selection from user filesystem. Requires `enctype="multipart/form-data"` on the parent `<form>` tag.

HTML: Complete University Course Registration Form

```

1  <form action="/api/register" method="POST"
    enctype="multipart/form-data">
2      <!-- Text and Password Inputs -->
3      <label for="fullname">Full Name:</label><br>
4      <input type="text" id="fullname" name="student_name"
        placeholder="Enter full name" required><br><br>
5
6      <label for="pwd">Password:</label><br>
7      <input type="password" id="pwd" name="password"
        required><br><br>
8
9      <!-- Radio Buttons: Mutual Exclusion via Shared Name -->
10     <label>Gender:</label><br>
11     <input type="radio" id="male" name="gender" value="male"
        checked>
12     <label for="male">Male</label>
13     <input type="radio" id="female" name="gender" value="female">
14     <label for="female">Female</label><br><br>
15
16     <!-- Checkboxes -->
17     <label>Course Electives:</label><br>
18     <input type="checkbox" id="c1" name="electives" value="ai">
19     <label for="c1">Artificial Intelligence</label><br>
20     <input type="checkbox" id="c2" name="electives" value="web"
        checked>
21     <label for="c2">Web Development</label><br><br>
22
23     <!-- Dropdown Menu -->
24     <label for="dept">Academic Department:</label><br>
25     <select id="dept" name="department">
26         <option value="cse" selected>Computer Science &
        Engineering</option>
27         <option value="it">Information Technology</option>
28         <option value="ece">Electronics & Communication</option>

```

```
29     </select><br><br>
30
31     <!-- File Upload -->
32     <label for="doc">Upload ID Card Document:</label><br>
33     <input type="file" id="doc" name="id_document"><br><br>
34
35     <!-- Submission and Reset Buttons -->
36     <input type="submit" value="Complete Registration">
37     <input type="reset" value="Clear Form">
38 </form>
```

2.8 Exam Tips & Common Student Pitfalls

Unit 2 High-Yield Traps

- **Broken Radio Groups:** Giving different `name` attributes to radio buttons intended for the same question allows users to select multiple options simultaneously. Always match the `name` attribute across the group.
- **File Upload Failure:** Forgetting `enctype="multipart/form-data"` on the `<form>` tag causes browsers to send only the filename string rather than the binary file content.
- **Colspan vs. Rowspan Confusion:** `colspan` merges cells across vertical columns (horizontal span); `rowspan` merges cells across horizontal rows (vertical span).

2.9 Output Prediction & Debugging Challenges

Output Prediction: Ordered List Offsets and Table Spanning

Question: Predict the exact structure rendered:

```
1 <ol type="A" start="4">
2   <li>Alpha</li>
3   <li>Beta</li>
4 </ol>
5 <table border="1">
6   <tr><th colspan="2">Header</th></tr>
7   <tr><td>1</td><td>2</td></tr>
8 </table>
```

Answer & Tracing:

- The `<ol>` begins at letter index 4: D. Alpha, followed by E. Beta.
- The table has a top header cell spanning 2 columns with "Header", followed by row 2 containing two cells: 1 and 2.

Debugging Challenge: Fixing File Uploads and Radio Button Grouping**Buggy Code:**

```
1 <form action="upload.php" method="POST">
2   <input type="radio" name="g1" value="M"> Male
3   <input type="radio" name="g2" value="F"> Female
4   <input type="file" name="resume">
5   <input type="submit">
6 </form>
```

Diagnosis & Correction:

1. **Bug 1:** Radio buttons have different names (g1 vs g2), breaking mutual exclusion.
2. **Bug 2:** Missing `enctype="multipart/form-data"` will prevent binary file uploading.

Corrected Code:

```
1 <form action="upload.php" method="POST"
   enctype="multipart/form-data">
2   <input type="radio" name="gender" value="M"> Male
3   <input type="radio" name="gender" value="F"> Female
4   <input type="file" name="resume">
5   <input type="submit">
6 </form>
```

2.10 Graded Programming Practice Problems & Solutions

Unit 2 Practice Exercises

1. **Level 1 (Foundation):** Create a nested list structure representing a computer science syllabus hierarchy (Modules and Sub-topics).
2. **Level 1 (Foundation):** Build an HTML link that opens a PDF syllabus document in a new browser tab with tooltip text.
3. **Level 2 (Standard):** Write HTML code for an embedded video player featuring playback controls, a custom poster frame, and fallback formats.
4. **Level 2 (Standard):** Create a weekly university lecture timetable table using `<th>`, `<td>`, and `colspan` for lunch breaks.
5. **Level 3 (Exam Tier):** Construct an employee directory table using `rowspan` to group employees under departmental divisions.
6. **Level 3 (Exam Tier):** Design a complete student feedback questionnaire form containing text inputs, rating radio groups, interest checkboxes, a feedback textarea, and validation attributes.
7. **Level 4 (Challenge):** Build a complex student examination registration portal combining table structures, embedded media instructions, and multi-control forms.

2.10.1 Complete Step-by-Step Worked Solutions

Worked Solution: Solutions to Unit 2 Practice Problems

Solution to Problem 4 (Timetable with Colspan):

```

1 <table border="1" style="width:100%; text-align:center;">
2   <tr>
3     <th>Day</th>
4     <th>9:00 - 10:00</th>
5     <th>10:00 - 11:00</th>
6     <th>11:00 - 12:00</th>
7     <th>12:00 - 1:00</th>
8   </tr>
9   <tr>
10    <td>Monday</td>
11    <td>CSE326 (Web)</td>
12    <td>MTH165 (Math)</td>
13    <td rowspan="2" style="background:#eee;">LUNCH BREAK</td>
14    <td>INT108 (Python)</td>
15  </tr>
16  <tr>
17    <td>Tuesday</td>
18    <td>PHY101 (Physics)</td>
19    <td>CSE111 (IT)</td>
20    <td>Library Study</td>
21  </tr>
22 </table>

```

Solution to Problem 6 (Feedback Form):

```

1 <form action="/feedback" method="POST">
2   <label for="sname">Student Name:</label>
3   <input type="text" id="sname" name="student_name"
4     required><br><br>
5   <label>Course Rating:</label>
6   <input type="radio" id="r1" name="rating" value="5" checked>
7   <label for="r1">Excellent</label>
8   <input type="radio" id="r2" name="rating" value="4"> <label
9     for="r2">Good</label>
10  <input type="radio" id="r3" name="rating" value="3"> <label
11    for="r3">Average</label><br><br>
12  <label for="comments">Detailed Feedback:</label><br>
13  <textarea id="comments" name="comments" rows="4" cols="40"
14    placeholder="Your thoughts..."></textarea><br><br>
15  <input type="submit" value="Submit Feedback">
16 </form>

```

2.11 Multiple Choice Questions (MCQs)

1. Which attribute specifies that an anchor link opens in a new browser tab?

- (a) `target="_self"` (b) `target="_blank"` (c) `target="_parent"` (d) `target="_newtab"`

Answer: (b) — *_blank opens links in a new window/tab.*

2. What is the required form attribute for uploading files via HTTP POST?

- (a) `enctype="text/plain"` (b) `enctype="multipart/form-data"` (c) `method="FILE"`
(d) `upload="true"`

Answer: (b) — *Binary file uploads require `multipart/form-data`.*

3. Which table attribute merges a single cell horizontally across 3 columns?

- (a) `rowspan="3"` (b) `colspan="3"` (c) `merge="3"` (d) `col="3"`

Answer: (b) — *`colspan` merges cells across horizontal columns.*

4. What is the primary purpose of the `alt` attribute on an `<img>` tag?

- (a) Sets image border (b) Fallback text for accessibility (c) Changes image color (d)
Downloads image

Answer: (b) — *`alt` supplies fallback text for accessibility and broken links.*

5. How is mutual exclusivity achieved among multiple radio button inputs?

- (a) Matching `id` attributes (b) Matching `name` attributes (c) Matching `value` attributes
(d) Wrapping in `<div>`

Answer: (b) — *Radio buttons sharing the same `name` toggle mutually.*

6. Which list element is used for descriptive glossary lists?

- (a) `<ul>` (b) `<ol>` (c) `<dl>` (d) `<list>`

Answer: (c) — *`<dl>` defines description/glossary lists.*

7. Which HTTP form method appends data directly into the URL query string?

- (a) POST (b) GET (c) PUT (d) FETCH

Answer: (b) — *`GET` appends form data to the URL.*

8. What is the default starting index for `<ol type="A" start="3">`?

- (a) A (b) B (c) C (d) 3

Answer: (c) — *`start="3"` with `type="A"` begins at letter 'C'.*

9. Which attribute displays a thumbnail preview image before video playback?

- (a) `src` (b) `preview` (c) `poster` (d) `thumbnail`

Answer: (c) — *`poster` sets the video placeholder image.*

10. Which element defines the descriptive data inside a `<dl>` list?

- (a) `<dt>` (b) `<dd>` (c) `<li>` (d) `<data>`

Answer: (b) — *`<dd>` defines the description data.*

2.12 Self-Assessment Unit Test

Unit 2 Examination (25 Marks — 45 Minutes)**Section A: Short Answer Concepts ($3 \times 2 = 6$ Marks)**

1. Differentiate between HTTP GET and POST methods during form submissions.
2. Explain the function of `colspan` and `rowspan` attributes in HTML tables.
3. Why must all radio buttons in a question group share the same `name` attribute?

Section B: Code Tracing & Output Prediction ($3 \times 3 = 9$ Marks)

4. Predict the output of this HTML list snippet:

```
1 <ol type="I" start="5">
2   <li>Module 5</li>
3   <li>Module 6</li>
4 </ol>
5
```

5. Identify the errors in the following form definition:

```
1 <form action="save.php" method="GET">
2   <input type="file" name="doc">
3   <input type="submit">
4 </form>
5
```

6. Explain the role of the `poster` attribute in HTML5 video elements.

Section C: Comprehensive Programming Problem ($1 \times 10 = 10$ Marks)

7. Write a complete HTML document for a University Student Course Admission Form:
 - (a) Collect candidate name, email, password, and phone number with validation attributes. (3 Marks)
 - (b) Provide a department dropdown (`<select>`), gender radio buttons, and hobby checkboxes. (3 Marks)
 - (c) Include a passport photo upload input and submit/reset buttons with proper form encoding. (4 Marks)

2.12.1 Unit Test Complete Solutions & Rubric**Worked Solution: Complete Solutions for Unit 2 Test****Section A Solutions:**

1. **GET vs. POST** [2 Marks]: GET appends form data to the URL (visible, limited length, bookmarkable); POST sends data in the HTTP request body (secure, large payloads, file uploads).
2. **Colspan vs. Rowspan** [2 Marks]: `colspan` merges cells across horizontal columns; `rowspan` merges cells across vertical rows.
3. **Radio Button Grouping** [2 Marks]: A shared `name` attribute informs the browser

that the inputs belong to the same logical group, enforcing mutual exclusion (only one selectable at a time).

Section B Solutions:

4. **List Tracing** [3 Marks]: `type="I"` and `start="5"` renders Roman numerals starting at 5: V. Module 5, followed by VI. Module 6.
5. **File Upload Form Error** [3 Marks]: File inputs cannot send binary payloads over GET without `enctype`. Fix: Change method to POST and add `enctype="multipart/form-data"`.
6. **Poster Attribute** [3 Marks]: Specifies a placeholder image URL displayed in the video frame before the user clicks play or while the video buffers.

Section C Solution:

7. **Complete Admission Form Implementation** [10 Marks]:

```

1  < DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>University Admission Portal</title>
6  </head>
7  <body>
8      <h2>Student Admission Registration</h2>
9      <form action="/admissions/submit" method="POST"
10         enctype="multipart/form-data">
11          <label for="name">Candidate Name:</label><br>
12          <input type="text" id="name" name="cand_name"
13             required><br><br>
14          <label for="email">Email Address:</label><br>
15          <input type="email" id="email" name="cand_email"
16             required><br><br>
17          <label for="pwd">Portal Password:</label><br>
18          <input type="password" id="pwd" name="cand_pwd"
19             required><br><br>
20          <label>Gender:</label><br>
21          <input type="radio" id="m" name="gender" value="male"
22             checked> <label for="m">Male</label>
23          <input type="radio" id="f" name="gender"
24             value="female"> <label for="f">Female</label><br><br>
25          <label for="dept">Engineering Stream:</label><br>
26          <select id="dept" name="stream">
27              <option value="cse">B.Tech Computer
28              Science</option>
29              <option value="it">B.Tech Information
30              Technology</option>
31              <option value="ai">B.Tech AI & Data
32              Science</option>
33          </select><br><br>

```

```
30         <label for="photo">Passport Photograph:</label><br>
31         <input type="file" id="photo" name="passport_photo"
           required><br><br>
32
33         <input type="submit" value="Register Now">
34         <input type="reset" value="Reset Form">
35     </form>
36 </body>
37 </html>
```

Marking Scheme: Inputs and validations: 3M; Radio, checkboxes, select: 3M; File upload and form headers: 4M.

Introduction to Cascading Style Sheets (CSS)

3.1 Unit Overview & Learning Objectives

Unit 3 Academic Scope

LPU CSE326

Cascading Style Sheets (CSS) govern the visual presentation, typography, and layout of HTML documents. This unit introduces CSS core philosophies, insertion methods (Inline, Internal, External), the specificity cascade, fundamental selector types (Element, ID, Class), typography and text control properties, the CSS Box Model (Content, Padding, Border, Margin), background imagery, and professional styling techniques for HTML tables and interactive forms.

- **CSS Architecture:** Master the separation of presentation from content, understanding the maintenance and caching benefits of external style sheets.
- **Insertion Modalities:** Compare inline, internal, and external CSS insertion rules and cascading priority conflicts.
- **Selector Mechanics:** Select elements precisely using type, ID (#), and reusable class (.) selectors.
- **Typography & Text Control:** Style web text using font stacks (`font-family`), relative/absolute size units, weights, alignments, transformations, and decorations.
- **The CSS Box Model:** Understand dimensional calculations, padding, margin collapsing, and border models.
- **Component Styling:** Style responsive tables with hover states and format interactive form inputs with dynamic focus rings.

3.2 Introduction to CSS & Separation of Concerns

3.2.1 What is CSS?

****Cascading Style Sheets (CSS)**** is a stylesheet language used to describe the presentation, layout, and visual formatting of documents authored in HTML.

3.2.2 Why Use CSS?

1. **Separation of Concerns:** HTML defines semantic document structure; CSS governs visual presentation.

2. **Site-wide Maintenance Efficiency:** Modifying a single external CSS file instantly updates styles across hundreds of webpages.
3. **Bandwidth Optimization:** External stylesheets are cached by browsers upon first download, drastically reducing server bandwidth and accelerating subsequent page loads.

3.3 CSS Insertion Methods (Types of CSS)

Insertion Type	Syntax & Implementation Location	Scope	Priority
Inline CSS	<code>style="..."</code> attribute directly on the HTML tag	Single element	**Highest** (Overrides others)
Internal CSS	<code><style></code> tag declared inside document <code><head></code>	Single HTML document	Medium
External CSS	<code><link rel="stylesheet" href="style.css"></code> in <code><head></code>	Multi-page website	**Lowest** (Base styles)

Table 3.1: Comparison of the Three CSS Insertion Methods.

3.3.1 1. Inline CSS

HTML: Inline CSS Example

```
1 <p style="color: blue; font-size: 16px; font-weight: bold;">
2   This paragraph is styled via inline CSS.
3 </p>
```

3.3.2 2. Internal CSS

HTML: Internal CSS in Document Head

```
1 <head>
2   <style>
3     body {
4       background-color: #f8fafc;
5     }
6     h1 {
7       color: #0284c7;
8       text-align: center;
9     }
10  </style>
11 </head>
```

3.3.3 3. External CSS (Best Practice)

HTML: Linking External Stylesheet

```
1 <head>
2     <title>University Portal</title>
3     <link rel="stylesheet" type="text/css" href="styles.css">
4 </head>
```

CSS: External File (styles.css)

```
1 /* Global site styles */
2 body {
3     font-family: Arial, sans-serif;
4     line-height: 1.6;
5     color: #333333;
6 }
```

3.4 CSS Selectors (Type, ID & Class)

A **CSS Selector** designates which HTML element(s) the style declarations will be applied to:

3.4.1 1. Type (Element) Selector

Matches all HTML elements sharing the specified tag name:

CSS: Type Selector

```
1 /* Matches all <p> elements on the page */
2 p {
3     color: #1e293b;
4     font-size: 14px;
5 }
```

3.4.2 2. ID Selector (#)

Matches the unique HTML element bearing the corresponding `id` attribute:

CSS: ID Selector

```
1 /* Matches <div id="main-header"> */
2 #main-header {
3     background-color: #0f172a;
4     color: #ffffff;
5     padding: 20px;
6 }
```

3.4.3 3. Class Selector (.)

Matches all HTML elements containing the specified `class` attribute name (reusable across multiple elements):

CSS: Class Selector

```
1  /* Matches any element with class="highlight" */
2  .highlight {
3      background-color: #fef08a;
4      font-weight: bold;
5      border-left: 4px solid #ca8a04;
6  }
```

3.5 Typography & Text Controlling Properties

CSS: Typography and Text Styling Example

```
1  h1 {
2      font-family: "Helvetica Neue", Helvetica, Arial, sans-serif;
3      font-size: 28px;
4      font-weight: bold;
5      text-align: center;
6      text-transform: uppercase;
7      color: #0f172a;
8  }
9
10 p {
11     font-size: 16px;
12     line-height: 1.75;
13     text-align: justify;
14 }
15
16 a.nav-link {
17     text-decoration: none; /* Removes default browser link
18     underline */
19     color: #0284c7;
20 }
```

3.6 The CSS Box Model

Every rendered HTML element is modeled as a rectangular geometric box composed of four concentric layers:

3.6.1 Box Model Layers (Inside Out)

1. **Content Box:** The area where actual text, images, and child nodes reside. Dimensions set via `width` and `height`.
2. **Padding:** Transparent space clearing the area between the content and the border.
3. **Border:** A visible line surrounding the padding and content.
4. **Margin:** Transparent space outside the border separating the element from adjacent elements.

CSS Property	Values	Visual Behavior	Example Code
<code>font-family</code>	Font stack list	Sets font type-face with fall-backs	<code>font-family: "Segoe UI", Arial, sans-serif;</code>
<code>font-size</code>	px, em, rem, %	Determines character height	<code>font-size: 18px;</code>
<code>font-weight</code>	normal, bold, 100–900	Sets stroke thickness	<code>font-weight: 700;</code>
<code>text-align</code>	left, right, center, justify	Aligns text horizontally	<code>text-align: center;</code>
<code>line-height</code>	Unitless number, px, %	Controls vertical space between lines	<code>line-height: 1.6;</code>
<code>text-decoration</code>	none, underline, line-through	Controls text underlines/strikes	<code>text-decoration: none;</code>
<code>text-transform</code>	uppercase, lowercase, capitalize	Alters letter case dynamically	<code>text-transform: uppercase;</code>

Table 3.2: Core Typography and Text Formatting Properties.

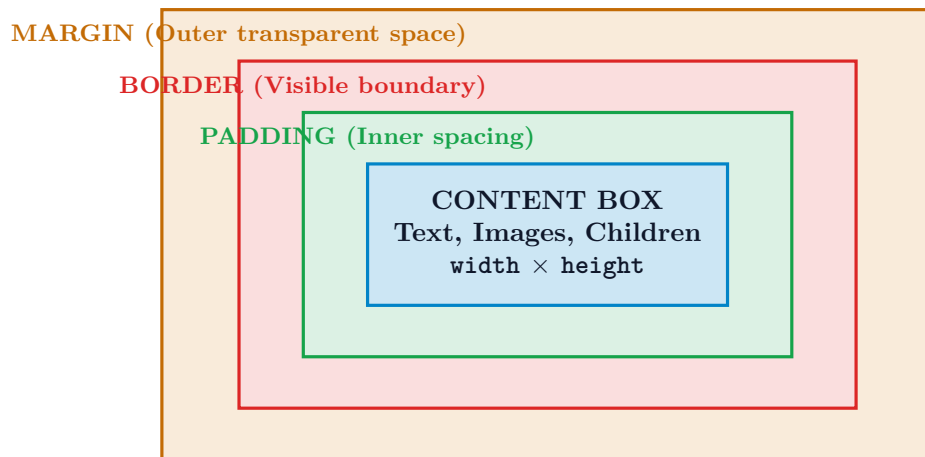


Figure 3.1: The Concentric Layers of the CSS Box Model.

3.6.2 Box Model Shorthand Notation

Syntax Specification: Padding and Margin Shorthand

```
/* 4 Values: Top Right Bottom Left (Clockwise) */ margin: 10px 20px 10px 20px;
/* 2 Values: Top/Bottom Left/Right */ padding: 15px 30px;
/* 1 Value: All 4 Sides */ margin: 20px;
```

CSS: Box Model in Practice

```
1 .card {
2   width: 320px;
3   padding: 20px;
4   border: 2px solid #cbd5e1;
5   border-radius: 8px;
6   margin: 25px auto; /* Centered horizontally */
7   box-sizing: border-box; /* Includes padding & border in total
8   320px width */
9 }
```

3.7 Working with Background Images

CSS: Background Image Configuration

```
1 .hero-banner {
2   background-color: #0f172a; /* Fallback color */
3   background-image: url("campus_hero.jpg");
4   background-repeat: no-repeat; /* repeat, no-repeat, repeat-x,
5   repeat-y */
6   background-position: center center; /* Horizontal Vertical */
7   background-size: cover; /* cover (scales to fill) or contain
8   (scales to fit) */
9   height: 400px;
10 }
```

3.8 Applying CSS on Tables and Forms

3.8.1 Professional Table Styling

CSS: Clean Styled Table with Hover States

```
1 table.styled-table {
2     width: 100%;
3     border-collapse: collapse; /* Merges adjacent cell borders
4     margin: 20px 0;
5     font-size: 15px;
6 }
7
8 table.styled-table th {
9     background-color: #0284c7;
10    color: #ffffff;
11    padding: 12px 15px;
12    text-align: left;
13 }
14
15 table.styled-table td {
16     padding: 12px 15px;
17     border-bottom: 1px solid #e2e8f0;
18 }
19
20 /* Zebra Striping */
21 table.styled-table tbody tr:nth-child(even) {
22     background-color: #f8fafc;
23 }
24
25 /* Hover Highlight */
26 table.styled-table tbody tr:hover {
27     background-color: #e0f2fe;
28     cursor: pointer;
29 }
```

3.8.2 Professional Form Styling

CSS: Modern Form Input and Button Styling

```
1 /* Input Fields */
2 input[type="text"],
3 input[type="email"],
4 input[type="password"],
5 select,
6 textarea {
7     width: 100%;
8     padding: 10px 14px;
9     margin: 8px 0 16px 0;
10    border: 1px solid #cbd5e1;
11    border-radius: 6px;
12    box-sizing: border-box; /* Crucial for full-width inputs */
13    font-size: 14px;
```

```
14 }
15
16 /* Dynamic Focus Highlight */
17 input[type="text"]:focus,
18 input[type="email"]:focus,
19 input[type="password"]:focus {
20     border-color: #0284c7;
21     outline: 2px solid rgba(2, 132, 199, 0.2);
22 }
23
24 /* Submit Buttons */
25 button.btn-primary,
26 input[type="submit"] {
27     background-color: #16a34a;
28     color: #ffffff;
29     padding: 12px 24px;
30     border: none;
31     border-radius: 6px;
32     font-weight: bold;
33     cursor: pointer;
34 }
35
36 button.btn-primary:hover,
37 input[type="submit"]:hover {
38     background-color: #15803d;
39 }
```

3.9 Exam Tips & Common Student Pitfalls

Unit 3 High-Yield Traps

- **ID Specificity Trap:** ID selectors (`#id`) have higher specificity than class selectors (`.class`), which in turn override element selectors (`p`).
- **Missing `border-collapse`:** By default, HTML tables display a 2px gap between cell borders. To achieve a modern single border, you **MUST** set `border-collapse: collapse;`
- **Box Model Width Expansion:** Without `box-sizing: border-box`, adding `padding: 20px` to a `width: 300px` box expands its actual rendered width to $300 + 20(2) = 340\text{px}$.

3.10 Output Prediction & Debugging Challenges

Output Prediction: CSS Specificity and Cascading Order

Question: What color will the paragraph text be rendered in?

```
1 <style>
2   p { color: blue; }
3   .text-green { color: green; }
4   #text-red { color: red; }
5 </style>
6 <p id="text-red" class="text-green" style="color: purple;">Hello
   CSS</p>
```

Answer & Specificity Tracing:

- Specificity hierarchy: Inline Style (1000) > ID Selector (100) > Class Selector (10) > Element Selector (1).
- The inline style `style="color: purple;"` has the highest specificity (1000).
- The text will render in **purple**.

Debugging Challenge: Fixing Box Model Overflow and Collapsing Borders

Buggy CSS:

```
1 table {
2   border: 1px solid black;
3   /* Table displays double borders! */
4 }
5 input[type="text"] {
6   width: 100%;
7   padding: 15px;
8   /* Input overflows its parent container! */
9 }
```

Diagnosis & Correction:

1. Add `border-collapse: collapse;` to `table` to merge double borders.
2. Add `box-sizing: border-box;` to `input` to prevent padding from expanding width beyond 100%.

3.11 Graded Programming Practice Problems & Solutions

Unit 3 Practice Exercises

1. **Level 1 (Foundation):** Write external CSS rules to style a university webpage with a centered `<h1>`, justified paragraphs, and navy blue headings.
2. **Level 1 (Foundation):** Style an unstyled list of links into a horizontal navigation bar.
3. **Level 2 (Standard):** Design a responsive card component with padding, border radius, subtle box shadow, and hover elevation.

4. **Level 2 (Standard):** Write CSS to create a zebra-striped table with dark headers and row hover highlights.
5. **Level 3 (Exam Tier):** Create a login form layout with stylized inputs, focused outline rings, and animated green submit buttons.
6. **Level 3 (Exam Tier):** Write CSS creating a full-width hero header with a centered background image using `background-size: cover`.
7. **Level 4 (Challenge):** Build a complete responsive university faculty profile layout combining Box Model cards, typography hierarchy, and styled contact tables.

3.11.1 Complete Step-by-Step Worked Solutions

Worked Solution: Solutions to Unit 3 Practice Problems

Solution to Problem 3 (Elevated Card Component):

```

1 .faculty-card {
2     width: 300px;
3     background-color: #ffffff;
4     border: 1px solid #e2e8f0;
5     border-radius: 8px;
6     padding: 20px;
7     margin: 15px;
8     box-sizing: border-box;
9     box-shadow: 0 4px 6px rgba(0, 0, 0, 0.05);
10    transition: transform 0.2s ease, box-shadow 0.2s ease;
11 }
12
13 .faculty-card:hover {
14     transform: translateY(-4px);
15     box-shadow: 0 10px 15px rgba(0, 0, 0, 0.1);
16 }

```

Solution to Problem 5 (Styled Login Form):

```

1 .login-box {
2     max-width: 360px;
3     margin: 50px auto;
4     padding: 30px;
5     border-radius: 8px;
6     background: #ffffff;
7     box-shadow: 0 4px 12px rgba(0,0,0,0.1);
8 }
9
10 .login-box input[type="text"],
11 .login-box input[type="password"] {
12     width: 100%;
13     padding: 10px 12px;
14     margin: 8px 0 16px;
15     border: 1px solid #cbd5e1;
16     border-radius: 4px;
17     box-sizing: border-box;
18 }
19
20 .login-box input:focus {

```

```
21     border-color: #0284c7;
22     outline: 2px solid rgba(2, 132, 199, 0.2);
23 }
24
25 .login-box button {
26     width: 100%;
27     padding: 12px;
28     background-color: #16a34a;
29     color: #ffffff;
30     border: none;
31     border-radius: 4px;
32     font-weight: bold;
33     cursor: pointer;
34 }
```

3.12 Multiple Choice Questions (MCQs)

1. Which CSS insertion method has the HIGHEST cascade specificity priority?

- (a) External CSS (b) Internal CSS (c) Inline CSS (d) Browser Default

Answer: (c) — *Inline CSS applied via `style="..."` has the highest priority.*

2. Which selector syntax targets all elements with `class="btn"`?

- (a) `#btn` (b) `.btn` (c) `*btn` (d) `btn`

Answer: (b) — *Class selectors are prefixed with a period (.).*

3. In the CSS Box Model, what is the layer immediately surrounding the content box?

- (a) Margin (b) Border (c) Padding (d) Outline

Answer: (c) — *Padding is the inner space directly surrounding the content.*

4. Which CSS property removes default underlines from hyperlinks?

- (a) `font-style: none` (b) `text-decoration: none` (c) `text-underline: off` (d)
`link-style: none`

Answer: (b) — *`text-decoration: none`; eliminates link underlines.*

5. What is the effect of `border-collapse: collapse;` on HTML tables?

- (a) Hides the table (b) Merges double borders into a single border (c) Adds padding
(d) Centers text

Answer: (b) — *`border-collapse: collapse`; eliminates gaps between adjacent cells.*

6. Which background property scales an image to cover the entire container?

- (a) `background-size: cover` (b) `background-size: contain` (c) `background-repeat: all`
(d) `background-fill: true`

Answer: (a) — *`background-size: cover`; scales the image to completely fill the element.*

7. What does `margin: 10px 20px;` specify?

- (a) Top/Bottom 10px, Left/Right 20px (b) Top 10px, Right 20px (c) All sides 10px
(d) Left 10px, Right 20px

Answer: (a) — *2-value shorthand sets Vertical (10px) and Horizontal (20px).*

8. Which property ensures padding is included within the specified `width`?

- (a) `box-sizing: border-box` (b) `box-sizing: content-box` (c) `display: block`
(d) `padding-mode: inside`

Answer: (a) — *border-box includes padding and border in total width/height.*

9. Which selector matches a single unique element with `id="footer"`?

- (a) `.footer` (b) `#footer` (c) `footer` (d) `@footer`

Answer: (b) — *ID selectors are prefixed with a hash (#).*

10. What is the pseudo-class used to style an element when clicked/focused?

- (a) `:hover` (b) `:focus` (c) `:active` (d) `:visited`

Answer: (b) — *:focus activates when an input element receives focus.*

3.13 Self-Assessment Unit Test

Unit 3 Examination (25 Marks — 45 Minutes)

Section A: Short Answer Concepts ($3 \times 2 = 6$ Marks)

1. Explain the advantages of External CSS over Inline CSS.
2. Describe the four concentric layers of the CSS Box Model.
3. Differentiate between the `id` and `class` selectors.

Section B: Code Tracing & Analysis ($3 \times 3 = 9$ Marks)

4. Given `width: 250px; padding: 15px; border: 5px solid black;`, compute the total rendered element width under (a) `box-sizing: content-box` and (b) `box-sizing: border-box`.

5. Predict the output color of the text:

```
1 <style>
2   p.intro { color: blue; }
3   p { color: red; }
4 </style>
5 <p class="intro">Welcome</p>
6
```

6. Explain how `tr:hover` creates hoverable interactive tables.

Section C: Comprehensive Programming Problem ($1 \times 10 = 10$ Marks)

7. Design a complete CSS stylesheet for an Online Course Information Card:
 - (a) Set card width, border radius, padding, and subtle box shadow with hover elevation. (4 Marks)
 - (b) Style a course title heading, an image banner, and a price tag. (3 Marks)
 - (c) Include a full-width call-to-action button with hover color transitions. (3 Marks)

3.13.1 Unit Test Complete Solutions & Rubric

Worked Solution: Complete Solutions for Unit 3 Test

Section A Solutions:

1. **External CSS Advantages** [2 Marks]: Promotes clean separation of concerns, enables site-wide reusability, simplifies updates, and reduces network bandwidth via browser caching.
2. **Box Model Layers** [2 Marks]: (1) Content (inner text/images); (2) Padding (transparent inner spacing); (3) Border (visible perimeter line); (4) Margin (transparent external spacing).
3. **ID vs. Class** [2 Marks]: ID (#) is unique and targets exactly one element per document. Class (.) is reusable and targets multiple elements across the page.

Section B Solutions:

4. **Box Model Calculations** [3 Marks]:
 - (a) **content-box**: Total Width = width + 2(padding) + 2(border) = 250 + 2(15) + 2(5) = **290px**.
 - (b) **border-box**: Total Width = **250px** (padding and border are absorbed inside the 250px boundary).
5. **Class Specificity Tracing** [3 Marks]: Class selector **p.intro** (specificity (11)) overrides type selector **p** (specificity (1)). Text color will be **blue**.
6. **Hoverable Table Logic** [3 Marks]: **tr:hover { background-color: #... }** listens for mouse enter events on each row, dynamically altering row background color to provide visual feedback.

Section C Solution:

7. **Course Card CSS Implementation** [10 Marks]:

```

1  .course-card {
2      width: 320px;
3      background-color: #ffffff;
4      border: 1px solid #e2e8f0;
5      border-radius: 10px;
6      padding: 20px;
7      margin: 20px auto;
8      box-sizing: border-box;
9      box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
10     transition: transform 0.2s ease, box-shadow 0.2s ease;
11 }

```

```
12
13 .course-card:hover {
14     transform: translateY(-6px);
15     box-shadow: 0 12px 20px -2px rgba(0, 0, 0, 0.15);
16 }
17
18 .course-card .title {
19     font-size: 20px;
20     font-weight: bold;
21     color: #0f172a;
22     margin-bottom: 8px;
23 }
24
25 .course-card .price {
26     font-size: 18px;
27     color: #16a34a;
28     font-weight: 600;
29     margin: 12px 0;
30 }
31
32 .course-card .btn-enroll {
33     display: block;
34     width: 100%;
35     padding: 12px;
36     background-color: #0284c7;
37     color: #ffffff;
38     text-align: center;
39     border: none;
40     border-radius: 6px;
41     font-weight: bold;
42     cursor: pointer;
43     transition: background-color 0.2s ease;
44 }
45
46 .course-card .btn-enroll:hover {
47     background-color: #0369a1;
48 }
```

Marking Scheme: Box model card styling & hover: 4M; Typography and price hierarchy: 3M; Button styling with hover state: 3M.

JavaScript Application Development (Syntax, Variables, Logic & Objects)

4.1 Unit Overview & Learning Objectives

Unit 4 Academic Scope

LPU CSE326

JavaScript provides the dynamic, programmable behavioral layer of the web. This unit covers script placement strategies within HTML documents, variable declarations and scope architectures (**var**, **let**, **const**), arithmetic, comparison, and logical operators, branching control structures (**if-else**, **switch**), iterative loops (**for**, **while**, **do-while**), browser popup dialogs (**alert**, **confirm**, **prompt**), and JavaScript object literal data structures with methods and the **this** keyword.

- **Script Architecture:** Compare script placement within `<head>` versus the bottom of `<body>`, and configure external `.js` files for caching.
- **Variable Scope & Lifecycle:** Contrast function-scoped **var** (and hoisting) with block-scoped **let** and **const** (Temporal Dead Zone).
- **Operator Mechanics:** Distinguish loose equality (`==`) from strict type-and-value equality (`===`).
- **Control Flow & Branching:** Implement multi-way decisions via **switch-case** and control loop iterations using **break** and **continue**.
- **Iterative Loops:** Design terminating **for**, **while**, and **do-while** loops.
- **Object-Oriented JavaScript:** Construct object literals, access properties via dot and bracket notation, and implement methods bound to **this**.

4.2 Incorporating JavaScript in HTML

JavaScript can be embedded into an HTML document using three distinct strategies:

HTML: Script Inclusion at Bottom of Body

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>JavaScript Integration</title>
6 </head>
```

Placement Strategy	Execution Timing & Characteristics	Recommended Use Case
Inside <head>	Loads and parses before the HTML body is rendered; blocks HTML parsing.	Pre-loading utilities, polyfills
Inside <body> (Bottom)	Executes after DOM elements are constructed; ensures non-blocking UI rendering.	**Standard Web Best Practice**
External .js File	Linked via <script src="app.js"></script>; browser caches script file.	Production multi-page applications

Table 4.1: JavaScript Inclusion Modalities in HTML Documents.

```
7 <body>
8   <h1 id="headline">Original Text</h1>
9   <button id="btnAction">Click to Modify</button>
10
11   <!-- External Script loaded at bottom of body -->
12   <script src="main.js"></script>
13 </body>
14 </html>
```

4.3 Variables: var, let, and const

Modern ECMAScript (ES6+) provides three variable declaration keywords with fundamentally different scoping and mutability rules:

Keyword	Scope	Hoisting Behavior	Re-declaration?	Reassignment
var	Function / Global	Hoisted & initialized as <code>undefined</code>	**Allowed**	**Allowed**
let	**Block Scope** (<code>{}</code>)	Hoisted in **Temporal Dead Zone (TDZ)**	Prohibited	**Allowed**
const	**Block Scope** (<code>{}</code>)	Hoisted in **Temporal Dead Zone (TDZ)**	Prohibited	**Prohibited** (Im-mutable bind-ing)

Table 4.2: Detailed Architectural Comparison of `var`, `let`, and `const`.

JavaScript: Scoping and Temporal Dead Zone Demonstration

```
1 function testScope() {
2   // var is function-scoped
3   if (true) {
4     var functionScoped = "Visible outside this if-block";
5     let blockScoped = "Visible ONLY inside this if-block";
6     const constantVal = 3.14159;
7   }
```

```

8     console.log(functionScoped); // Works: "Visible outside this
    if-block"
9     // console.log(blockScoped); // ReferenceError: blockScoped
    is not defined
10    // console.log(constantVal); // ReferenceError: constantVal
    is not defined
11 }
12
13 // Mutability in const objects
14 const student = { name: "Rohan", gpa: 8.5 };
15 student.gpa = 9.0; // Allowed: Modifying property of object
16 console.log("Updated GPA:", student.gpa);
17
18 // student = { name: "New" }; // TypeError: Assignment to
    constant variable

```

Browser Rendered Output / Console Display

Visible outside this if-block Updated GPA: 9.0

4.4 Operators & Type Coercion

4.4.1 Arithmetic and Assignment Operators

JavaScript supports standard arithmetic operators: + (addition / string concatenation), - (subtraction), * (multiplication), / (true division), % (modulus), ++ (increment), and -- (decrement).

4.4.2 Loose Equality (==) vs. Strict Equality (===)

The Critical Strict Equality Rule

HIGH YIELD

- **Loose Equality (==):** Performs **implicit type coercion** before comparing values. `5 == "5"` evaluates to `True` because the string "5" is coerced to number 5.
- **Strict Equality (===):** Evaluates equality of **both value and data type** without coercion. `5 === "5"` evaluates to `False` (number vs. string).

JavaScript: Loose vs Strict Equality Comparisons

```

1 console.log(5 == "5");      // true (Type coercion applied)
2 console.log(5 === "5");    // false (Distinct data types: Number
    vs String)
3 console.log(0 == false);    // true (Coerced to 0)
4 console.log(0 === false);   // false (Number vs Boolean)
5 console.log(null == undefined); // true
6 console.log(null === undefined); // false

```

4.4.3 Logical Operators and Short-Circuit Evaluation

- **Logical AND (&&):** Returns `true` if both operands are truthy. Short-circuits on first falsy value.

- **Logical OR (||)**: Returns `true` if at least one operand is truthy. Short-circuits on first truthy value.
- **Logical NOT (!)**: Inverts truth value (`!true → false`).

4.5 Control Flow & Looping Statements

4.5.1 The `if...else` and `switch` Statements

JavaScript: Multi-way Decision Tree via `switch`

```

1  let dayCode = 3;
2  let dayName = "";
3
4  switch (dayCode) {
5      case 1:
6          dayName = "Monday";
7          break;
8      case 2:
9          dayName = "Tuesday";
10         break;
11     case 3:
12         dayName = "Wednesday";
13         break;
14     default:
15         dayName = "Invalid Day Code";
16 }
17 console.log("Scheduled Day:", dayName);

```

4.5.2 Iterative Loops: `for`, `while`, and `do...while`

Loop Construct	Evaluation Timing & Mechanics	Guaranteed Iterations
<code>for (init; cond; step)</code>	Evaluates condition **before** each iteration (Pre-test)	0 or more
<code>while (condition)</code>	Evaluates condition **before** loop entry (Pre-test)	0 or more
<code>do { ... } while (cond)</code>	Executes body first, then evaluates condition (Post-test)	**At least 1 execution**

Table 4.3: Comparison of JavaScript Looping Statements.

JavaScript: Demonstrating `do...while` Guarantee

```

1  let count = 10;
2
3  // Condition (count < 5) is FALSE from the start
4  do {
5      console.log("Executed once regardless of condition! count =",
6          count);
7      count++;
8  } while (count < 5);

```

Browser Rendered Output / Console Display

Executed once regardless of condition! count = 10

4.6 Browser Popup Dialogs (alert, confirm, prompt)

Dialog Method	Return Value	Primary Purpose
alert("msg")	undefined	Displays informational notice with an "OK" button
confirm("msg")	Boolean (true on OK, false on Cancel)	Requests user binary confirmation for destructive actions
prompt("msg", "default")	String (Input text) or null (on Cancel)	Prompts user for a single-line textual string value

Table 4.4: JavaScript Browser Popup Dialog Functions.

JavaScript: Using Popup Dialogs

```

1  // 1. Alert
2  alert("Examination portal session initialized.");
3
4  // 2. Confirm
5  let deleteConfirm = confirm("Are you sure you wish to delete this
6    record?");
7  if (deleteConfirm) {
8    console.log("Record deleted.");
9  } else {
10   console.log("Deletion aborted by user.");
11 }
12
13 // 3. Prompt
14 let studentName = prompt("Enter student name:", "Aarav");
15 if (studentName !== null) {
16   console.log("Welcome,", studentName);
17 }

```

4.7 Working with JavaScript Objects

4.7.1 Object Literals, Properties & Methods

JavaScript: Object Creation, Methods, and the this Reference

```
1 // Object Literal Notation
2 const universityCourse = {
3   courseCode: "CSE326",
4   courseTitle: "Web Development",
5   credits: 3,
6   enrolledStudents: 120,
7
8   // Object Method
9   getCourseSummary: function() {
10     // 'this' references the host object instance
11     return `${this.courseCode}: ${this.courseTitle}
12     (${this.credits} Credits)`;
13   }
14 };
15
16 // Accessing properties via Dot and Bracket notation
17 console.log("Course Code (Dot):", universityCourse.courseCode);
18 console.log("Course Title (Bracket):",
19   universityCourse["courseTitle"]);
20
21 // Invoking Object Method
22 console.log("Summary:", universityCourse.getCourseSummary());
23
24 // Dynamically adding new property
25 universityCourse.instructor = "Dr. Sharma";
26 console.log("Instructor:", universityCourse.instructor);
```

Browser Rendered Output / Console Display

Course Code (Dot): CSE326 Course Title (Bracket): Web Development Summary:
CSE326: Web Development (3 Credits) Instructor: Dr. Sharma

4.8 Exam Tips & Common Student Pitfalls

Unit 4 High-Yield Traps

- **Missing break in Switch:** Forgetting `break` causes execution to "fall through" into subsequent cases regardless of whether their condition matches.
- **The `==` vs `===` Trap:** Always prefer `===` in examinations and production code to prevent unintended type coercions (e.g., `"" == 0` is `true`).
- **Reassigning `const` vs Mutating Object:** You CAN modify properties of a `const` object (`obj.x = 10`), but you CANNOT rebind the identifier to a new object (`obj = {}`).

4.9 Output Prediction & Debugging Challenges

Output Prediction: Variable Hoisting and Strict Equality

Question: Predict the exact output logged:

```
1 console.log(x); // Hoisting test
2 var x = 10;
3 console.log("5" + 2, "5" - 2);
4 console.log([] == 0, [] === 0);
```

Answer & Tracing:

- `var x` is hoisted and initialized with `undefined`. Line 1 logs `undefined`.
- `"5" + 2` performs string concatenation $\rightarrow$ `"52"`. `"5" - 2` coerces to numeric subtraction $\rightarrow$ `3`.
- `[] == 0` coerces empty array to empty string then to `0` $\rightarrow$ `true`. `[] === 0` compares Object vs Number $\rightarrow$ `false`.

Debugging Challenge: Fixing Scope Leakage and Switch Fallthrough

Buggy Code:

```
1 function compute(val) {
2   switch(val) {
3     case 1:
4       var res = "One";
5     case 2:
6       var res = "Two";
7   }
8   return res;
9 }
10 console.log(compute(1)); // Returns "Two" instead of "One"!
```

Diagnosis & Correction: Missing `break` causes case 1 to fall through into case 2. Use `let` and add `break`:

```
1 function compute(val) {
2   let res = "";
3   switch(val) {
4     case 1:
5       res = "One";
6       break;
7     case 2:
8       res = "Two";
9       break;
10  }
11  return res;
12 }
```

4.10 Graded Programming Practice Problems & Solutions

Unit 4 Practice Problems

1. **Level 1 (Foundation):** Write a JavaScript script that prompts the user for their birth year and calculates their current age using an alert dialog.
2. **Level 1 (Foundation):** Write a `for` loop that prints all even numbers between 1 and 20 to the browser console.
3. **Level 2 (Standard):** Create a `do...while` loop that simulates rolling a 6-sided die using `Math.random()` until a 6 is rolled.
4. **Level 2 (Standard):** Write a function that accepts a student exam percentage and returns letter grades using a `switch(true)` construct.
5. **Level 3 (Exam Tier):** Create a complete `bankAccount` object literal with properties `accountNumber`, `balance`, and methods `deposit(amount)`, `withdraw(amount)`, and `getBalance()`.
6. **Level 3 (Exam Tier):** Write a JavaScript program that calculates the factorial of an input number using a `while` loop.
7. **Level 4 (Challenge):** Design an interactive command-line quiz application utilizing objects, array iteration, `prompt()` user inputs, and score calculations.

4.10.1 Complete Step-by-Step Worked Solutions**Worked Solution: Solutions to Unit 4 Practice Problems****Solution to Problem 3 (Die Roll Simulation):**

```
1 let rollCount = 0;
2 let rollValue = 0;
3
4 do {
5     // Generate integer between 1 and 6
6     rollValue = Math.floor(Math.random() * 6) + 1;
7     rollCount++;
8     console.log('Roll ${rollCount}: Rolled a ${rollValue}');
9 } while (rollValue !== 6);
10
11 console.log('Target 6 reached in ${rollCount} rolls!');
```

Solution to Problem 5 (Bank Account Object):

```
1 const bankAccount = {
2     accountNumber: "LPU-CSE-9901",
3     balance: 5000.0,
4
5     deposit: function(amount) {
6         if (amount > 0) {
7             this.balance += amount;
8             return `Deposited Rs. ${amount}. New Balance: Rs.
9             ${this.balance}`;
10         }
11         return "Invalid deposit amount.";
12     },
13 }
```

```
12
13     withdraw: function(amount) {
14         if (amount > 0 && amount <= this.balance) {
15             this.balance -= amount;
16             return 'Withdrew Rs. ${amount}. Remaining Balance: Rs.
17             ${this.balance}';
18         }
19         return "Transaction Denied: Insufficient Funds.";
20     },
21     getBalance: function() {
22         return 'Current Balance: Rs. ${this.balance}';
23     }
24 };
25
26 console.log(bankAccount.deposit(1500));
27 console.log(bankAccount.withdraw(2000));
```

4.11 Multiple Choice Questions (MCQs)

1.

(a)

2.

(a)

3.

(a)

4.

(a)

5.

(a)

6.

(a)

7.

(a)

8.

(a)

9.

(a)

10.

(a)

4.12 Self-Assessment Unit Test

Unit 4 Examination (25 Marks — 45 Minutes)

Section A: Short Answer Concepts ($3 \times 2 = 6$ Marks)

1. Differentiate between `var`, `let`, and `const`.
2. Explain the difference between `==` and `===` in JavaScript.
3. What is the role of the `this` keyword in object methods?

Section B: Code Tracing & Output Prediction ($3 \times 3 = 9$ Marks)

4. Predict the output of this loop and operator code:

```
1 let sum = 0;
2 for (let i = 1; i <= 5; i++) {
3     if (i === 3) continue;
4     sum += i;
5 }
6 console.log("Sum:", sum);
7
```

5. Trace the output of this object method call:

```
1 const calc = {
2     num: 10,
3     double: function() { return this.num * 2; }
4 };
5 console.log(calc.double());
6
```

6. What is returned by `prompt("Enter score")` if the user types 95 and clicks OK vs. Cancel?

Section C: Comprehensive Programming Problem ($1 \times 10 = 10$ Marks)

7. Design a complete JavaScript interactive student grade evaluation system:
- Prompt user for 3 subject scores using `prompt()` and parse to numbers. (3 Marks)
 - Calculate average and assign letter grades using a `switch(true)` or chained `if-else`. (3 Marks)
 - Create a `reportCard` object storing name, scores, average, and a `printStatus()` method displayed via an `alert()`. (4 Marks)

4.12.1 Unit Test Complete Solutions & Rubric**Worked Solution: Complete Solutions for Unit 4 Test****Section A Solutions:**

- Var vs. Let vs. Const** [2 Marks]: `var` is function-scoped and hoisted with `undefined`; `let` is block-scoped and mutable; `const` is block-scoped with immutable binding.
- Equality Operators** [2 Marks]: `==` coerces operand types before comparing values; `===` compares value and type strictly without coercion.
- This Keyword** [2 Marks]: `this` refers to the calling object instance within whose context the method is executed.

Section B Solutions:

- Loop Tracing** [3 Marks]: $i = 1 \rightarrow \text{sum} = 1$; $i = 2 \rightarrow \text{sum} = 3$; $i = 3 \rightarrow \text{skip}$; $i = 4 \rightarrow \text{sum} = 7$; $i = 5 \rightarrow \text{sum} = 12$. Output: "Sum: 12".
- Object Method Output** [3 Marks]: `this.num` accesses 10, computing $10 \times 2 = 20$.
- Prompt Return Values** [3 Marks]: Clicking OK returns string "95"; clicking Cancel returns `null`.

Section C Solution:

7. **Interactive Grade Evaluation Script** [10 Marks]:

```

1 // Step 1: Collect user inputs
2 let studentName = prompt("Enter student name:", "Aarav");
3 let m1 = parseFloat(prompt("Enter marks for Subject 1
  (0-100):"));
4 let m2 = parseFloat(prompt("Enter marks for Subject 2
  (0-100):"));
5 let m3 = parseFloat(prompt("Enter marks for Subject 3
  (0-100):"));
6
7 // Step 2 & 3: Construct Report Object
8 const reportCard = {
9   name: studentName,
10  marks: [m1, m2, m3],
11
12  getAverage: function() {

```

```
13         return (this.marks[0] + this.marks[1] + this.marks[2])
14     / 3;
15     },
16     getGrade: function() {
17         let avg = this.getAverage();
18         if (avg >= 90) return "A+";
19         if (avg >= 80) return "A";
20         if (avg >= 70) return "B";
21         if (avg >= 60) return "C";
22         return "F";
23     },
24
25     displayReport: function() {
26         let summary = `--- Student Report Card ---\nName:
27         ${this.name}\nAverage:
28         ${this.getAverage().toFixed(2)}\nGrade: ${this.getGrade()}`;
29         alert(summary);
30     }
31 };
32 reportCard.displayReport();
```

Marking Scheme: Input parsing: 3M; Average and grade logic: 3M; Object structure and alert display: 4M.

JavaScript Functions, Events and Validation

5.1 Unit Overview & Learning Objectives

Unit 5 Academic Scope

LPU CSE326

Building interactive client-side web applications requires modular function design, responsive event-driven architectures, and defensive form validation. This unit covers function declarations, argument semantics (Pass-by-Value vs. Pass-by-Reference), the variable-length `arguments` object, event handling mechanisms (`Inline`, `DOM Property`, and `addEventListener`), mouse, keyboard, and form event lifecycles, and client-side data validation using conditional verification and regular expressions.

-
-
-
-

5.2 Working with Functions

5.2.1 Syntax and Declaration

Syntax Specification: Function Declaration

```
function functionName(parameter1, parameter2, ...)    // Function body logic return  
calculatedResult; // Optional return statement
```

Data Type Category	Passing Mechanism	Mutation Impact on Original Caller Variable
Primitive Types (number, string, boolean)	Pass-by-Value: A distinct copy of the data value is passed into the function stack frame.	Modifying the parameter inside the function does NOT affect the caller's variable.
Compound Types (Object, Array)	Pass-by-Reference: A copy of the memory reference pointer is passed.	Mutating object properties or array elements inside the function directly alters the original object in memory!

Table 5.1: Parameter Passing Modalities in JavaScript.

5.2.2 Pass-by-Value vs. Pass-by-Reference

JavaScript: Pass-by-Value vs Pass-by-Reference in Action

```

1 // 1. Pass-by-Value (Primitive)
2 let score = 50;
3 function updateScore(val) {
4     val = 100; // Local copy modified
5 }
6 updateScore(score);
7 console.log("Score after function call:", score); // 50
8     (Unchanged!)
9
10 // 2. Pass-by-Reference (Object)
11 let student = { name: "Ananya", marks: 80 };
12 function updateMarks(obj) {
13     obj.marks = 95; // Mutates original memory object
14 }
15 updateMarks(student);
16 console.log("Student marks after function call:", student.marks);
17     // 95 (Mutated!)

```

5.2.3 The arguments Object

JavaScript: Variable-Length Function Arguments

```

1 function sumAllNumbers() {
2     let total = 0;
3     // Iterate through all passed arguments dynamically
4     for (let i = 0; i < arguments.length; i++) {
5         total += arguments[i];
6     }
7     return total;
8 }
9

```

```

10 console.log("Sum (3 args):", sumAllNumbers(10, 20, 30)); //
    60
11 console.log("Sum (5 args):", sumAllNumbers(5, 10, 15, 20, 25));
    // 75

```

5.3 Working with JavaScript Events

5.3.1 The Three Event Handling Mechanisms

- 1.
- 2.
- 3.

5.3.2 Catalog of Core Web Events

Category	Event Name	Trigger Condition	Common Use Case
Mouse	click	User presses and releases mouse button	Button action, link override
	dblclick	User rapidly clicks element twice	Zooming, modal editing
	mouseover	Mouse cursor enters element boundary	Hover highlights, tooltip display
	mouseout	Mouse cursor leaves element boundary	Revert hover highlight
	mousemove	Mouse moves over element	Canvas drawing, coordinate tracking
Keyboard	keydown	User presses down any keyboard key	Game controls, keyboard shortcuts
	keyup	User releases keyboard key	Live character counting, search filter
Form	submit	User submits form	Client-side validation
	focus	Form input receives focus/cursor	Displaying active focus rings
	blur	Form input loses focus	Immediate single-field validation
	change	Input value committed (on blur/select)	Dropdown selection updates
	input	Value altered instantaneously	Real-time text search

Table 5.2: Comprehensive Classification of Core JavaScript Events.

JavaScript: Mouse and Keyboard Event Listeners

```

1 const box = document.getElementById("interactiveBox");
2 const inputField = document.getElementById("txtMessage");

```

```

3  const charCounter = document.getElementById("lblCount");
4
5  // Mouse Events
6  box.addEventListener("mouseover", function() {
7      box.style.backgroundColor = "#0284c7"; // Blue on hover
8  });
9
10 box.addEventListener("mouseout", function() {
11     box.style.backgroundColor = "#slate"; // Revert color
12 });
13
14 // Keyboard Event: Real-time Character Counter
15 inputField.addEventListener("keyup", function() {
16     charCounter.innerText = 'Characters typed:
17     ${inputField.value.length}';
18 });

```

5.4 Client-Side JavaScript Form Validation

5.4.1 The Validation Strategy

- 1.
- 2.
- 3.
- 4.

5.4.2 Essential Validation Techniques

-
-
-

—
—

HTML: HTML Form Markup for Validation

```

1  <form id="admissionForm" action="/submit" method="POST">
2      <label for="uname">Student Name (Min 4 chars):</label><br>
3      <input type="text" id="uname" name="username"><br><br>
4
5      <label for="uemail">Email Address:</label><br>
6      <input type="text" id="uemail" name="email"><br><br>
7
8      <label for="uage">Age (17 to 30):</label><br>

```

```

9      <input type="text" id="uage" name="age"><br><br>
10
11      <button type="submit">Submit Application</button>
12      <p id="errorBox" style="color: red; font-weight: bold;"></p>
13 </form>

```

JavaScript: Complete Client-Side Form Validation Logic

```

1  document.getElementById("admissionForm").addEventListener("submit",
    function(event) {
2      const nameVal = document.getElementById("uname").value.trim();
3      const emailVal =
document.getElementById("uemail").value.trim();
4      const ageVal = document.getElementById("uage").value.trim();
5      const errorBox = document.getElementById("errorBox");
6
7      let isValid = true;
8      let errorMessage = "";
9
10     // 1. Name Validation
11     if (nameVal === "") {
12         errorMessage = "Student Name is required.";
13         isValid = false;
14     } else if (nameVal.length < 4) {
15         errorMessage = "Student Name must be at least 4
characters.";
16         isValid = false;
17     }
18
19     // 2. Email Validation via Regex
20     const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
21     if (isValid && !emailRegex.test(emailVal)) {
22         errorMessage = "Please enter a valid email address.";
23         isValid = false;
24     }
25
26     // 3. Numeric Age Validation
27     const ageNum = parseInt(ageVal, 10);
28     if (isValid && (isNaN(ageNum) || ageNum < 17 || ageNum > 30))
{
29         errorMessage = "Age must be a valid number between 17 and
30.";
30         isValid = false;
31     }
32
33     // Decision: Block submission or allow
34     if (!isValid) {
35         event.preventDefault(); // Stop form from submitting!
36         errorBox.innerText = errorMessage;
37     } else {
38         errorBox.innerText = "";
39         alert("Validation Successful! Transmitting
application...");
40     }
41 });

```

5.5 Exam Tips & Common Student Pitfalls

Unit 5 High-Yield Traps

- **Forgetting `event.preventDefault()`:** In form validation, if you fail to call `event.preventDefault()`, the browser will proceed with page submission and reload even if validation failed!
- **Event Name Prefix:** In `addEventListener("click", fn)`, NEVER include the "on" prefix. Use "click", not "onclick".
- **isNaN() String Trap:** `isNaN("123")` returns `false` (because the string coerces to number 123). `isNaN("abc")` returns `true`.

5.6 Output Prediction & Debugging Challenges

Output Prediction: Arguments Object and Pass-by-Reference

Question: What will be logged to the console?

```
1 function modify(a, arr) {  
2     a = a * 2;  
3     arr.push(99);  
4 }  
5 let num = 10;  
6 let list = [1, 2];  
7 modify(num, list);  
8 console.log(num, list);
```

Answer & Tracing:

- `num` is primitive (pass-by-value); local modification does not affect outer variable → **10**.
- `list` is an array (pass-by-reference); pushing 99 mutates the original object in memory → **[1, 2, 99]**.
- Output: **10 [1, 2, 99]**

Debugging Challenge: Fixing Event Listener Syntax and Missing PreventDefault

Buggy Code:

```
1 form.addEventListener("onsubmit", function () {  
2     if (input.value == "") {  
3         alert("Empty field!");  
4         // Form submits anyway and page reloads!  
5     }  
6 });
```

Diagnosis & Correction:

1. **Bug 1:** Event name has "on" prefix. Change "onsubmit" to "submit".

2. **Bug 2:** Missing `event.preventDefault()` allows form submission.

Corrected Code:

```
1 form.addEventListener("submit", function(event) {
2     if (input.value.trim() === "") {
3         event.preventDefault();
4         alert("Empty field!");
5     }
6 });
```

5.7 Graded Programming Practice Problems & Solutions

Unit 5 Practice Exercises

1. **Level 1 (Foundation):** Write a JavaScript function that takes a temperature in Celsius and returns its Fahrenheit equivalent ($F = \frac{9}{5}C + 32$).
2. **Level 1 (Foundation):** Attach a click event listener to a button that toggles page background color between white and dark slate.
3. **Level 2 (Standard):** Write a function utilizing the `arguments` object to compute the mathematical average of an arbitrary number of inputs.
4. **Level 2 (Standard):** Create a live password character counter that turns red when password length is under 8 characters using `keyup`.
5. **Level 3 (Exam Tier):** Write a complete form validation script checking that passwords match in a "Confirm Password" registration field.
6. **Level 3 (Exam Tier):** Implement a telephone number validator that verifies 10-digit Indian mobile numbers starting with 6, 7, 8, or 9 using regex.
7. **Level 4 (Challenge):** Build an interactive registration form with real-time field-by-field blur validation and error visual indicators.

5.7.1 Complete Step-by-Step Worked Solutions

Worked Solution: Solutions to Unit 5 Practice Problems

Solution to Problem 3 (Average with arguments Object):

```
1 function calculateAverage() {
2     if (arguments.length === 0) return 0;
3     let sum = 0;
4     for (let i = 0; i < arguments.length; i++) {
5         sum += arguments[i];
6     }
7     return sum / arguments.length;
8 }
9
10 console.log("Average:", calculateAverage(10, 20, 30, 40)); // 25
```

Solution to Problem 5 (Password Match Validation):

```
1 document.getElementById("signupForm").addEventListener("submit",
  function(e) {
2     const p1 = document.getElementById("pass1").value;
3     const p2 = document.getElementById("pass2").value;
4     const err = document.getElementById("passError");
5
6     if (p1.length < 8) {
7         e.preventDefault();
8         err.innerText = "Password must be at least 8 characters.";
9         return;
10    }
11
12    if (p1 !== p2) {
13        e.preventDefault();
14        err.innerText = "Passwords do not match!";
15        return;
16    }
17    err.innerText = "";
18 });
```

5.8 Multiple Choice Questions (MCQs)

1.

(a)

2.

(a)

3.

(a)

4.

(a)

5.

(a)

6.

(a)

7.

(a)

8.

(a)

9.

(a)

10.

(a)

5.9 Self-Assessment Unit Test

Unit 5 Examination (25 Marks — 45 Minutes)

Section A: Short Answer Concepts ($3 \times 2 = 6$ Marks)

1. Explain the role of `event.preventDefault()` in form validation.
2. Differentiate between Pass-by-Value and Pass-by-Reference in JavaScript.
3. Why is `addEventListener()` preferred over DOM property event binding?

Section B: Code Tracing & Analysis ($3 \times 3 = 9$ Marks)

4. Predict the output:

```
1 function test(a, b) {  
2     console.log(arguments.length, a + b);  
3 }  
4 test(5, 10, 15, 20);  
5
```

5. Write a regular expression that validates a 10-digit mobile number starting with 7, 8, or 9.
6. Explain the difference between `input` and `change` events.

Section C: Comprehensive Programming Problem ($1 \times 10 = 10$ Marks)

7. Write a complete JavaScript form validation script for a User Registration Portal:
 - (a) Validate that username is non-empty and contains between 5 and 15 alphanumeric characters. (3 Marks)
 - (b) Validate email address format using regular expressions. (3 Marks)
 - (c) Validate that password is at least 8 characters long and matches the confirmation password field, displaying error messages in a dedicated <div>. (4 Marks)

5.9.1 Unit Test Complete Solutions & Rubric

Worked Solution: Complete Solutions for Unit 5 Test

Section A Solutions:

1. **PreventDefault** [2 Marks]: Halts the default browser behavior of form submission, preventing HTTP page navigation when client validation fails.
2. **Pass-by-Value vs. Reference** [2 Marks]: Primitives pass isolated copies of data values; objects and arrays pass memory reference pointers, allowing in-place property mutations.
3. **AddEventListener Benefits** [2 Marks]: Allows attaching multiple independent listener functions to a single event without overwriting existing handlers.

Section B Solutions:

4. **Arguments Tracing** [3 Marks]: `arguments.length` is 4; $a + b = 5 + 10 = 15$. Output: 4 15.
5. **Mobile Regex** [3 Marks]: `/^[7-9]\d{9}$/`.
6. **Input vs. Change** [3 Marks]: `input` fires synchronously on every single character change; `change` fires only when the user commits the value (e.g., clicking away/blur).

Section C Solution:

7. **Registration Form Validation Implementation** [10 Marks]:

```

1  document.getElementById("regForm").addEventListener("submit",
    function(e) {
2      const user =
document.getElementById("txtUser").value.trim();
3      const email =
document.getElementById("txtEmail").value.trim();
4      const pass1 = document.getElementById("txtPass1").value;
5      const pass2 = document.getElementById("txtPass2").value;
6      const errBox = document.getElementById("errDisplay");
7
8      let err = "";
9
10     // 1. Username Check (5-15 Alphanumeric)
11     const userRegex = /^[a-zA-Z0-9]{5,15}$/;
12     if (!userRegex.test(user)) {
13         err = "Username must be 5-15 alphanumeric characters.";
14     }
15     // 2. Email Check

```

```
16     else if (!/^[\s@]+@[\s@]+\.[\s@]+$/ .test(email)) {
17         err = "Please enter a valid email address.";
18     }
19     // 3. Password Length & Match
20     else if (pass1.length < 8) {
21         err = "Password must be at least 8 characters long.";
22     } else if (pass1 !== pass2) {
23         err = "Passwords do not match.";
24     }
25
26     if (err !== "") {
27         e.preventDefault(); // Stop submission
28         errBox.innerText = err;
29     } else {
30         errBox.innerText = "";
31         alert("Registration successfully validated!");
32     }
33 });
```

Marking Scheme: Username validation: 3M; Email regex validation: 3M; Password match & error display logic: 4M.

JavaScript Document Object Model (DOM)

6.1 Unit Overview & Learning Objectives

Unit 6 Academic Scope

LPU CSE326

The Document Object Model (DOM) is the language-neutral W3C standard application programming interface that represents HTML documents as hierarchical trees of objects. This unit covers the DOM tree architecture, the root `document` object, element selection methods (traditional versus modern CSS selectors), dynamic node creation, insertion, removal, tree traversal, content and attribute mutation (`innerHTML` vs. `textContent`), dynamic CSS manipulation via `style` and `classList`, the event object, and event propagation mechanisms (Bubbling vs. Capturing).

-
-
-
-
-
-

6.2 DOM Architecture & The Document Tree

Key Architectural Concepts of the DOM

- **Tree Structure:** Every element, attribute, and text snippet is represented as a `**Node**` in the tree.

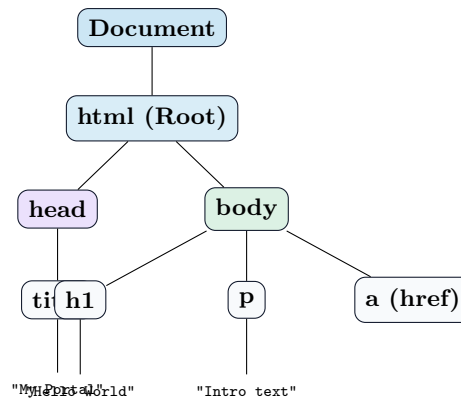


Figure 6.1: Hierarchical Document Object Model (DOM) Tree Representation.

- **Object-Oriented Interface:** Nodes expose properties (data states) and methods (callable actions).
- **Platform-Independent Standard:** Standardized by the W3C and WHATWG, ensuring cross-browser consistency.
- **Dynamic HTML (DHTML):** Enables scripts to add, modify, or delete elements instantaneously without triggering full page reloads.

6.3 The document Root Object

Document Property	Data Type	Description / Return Value
<code>document.title</code>	String	Gets or sets the document title displayed in the browser tab
<code>document.URL</code>	String	Returns the complete URL address of the loaded document
<code>document.body</code>	Element	Direct reference to the <code><body></code> container node
<code>document.head</code>	Element	Direct reference to the <code><head></code> metadata container node
<code>document.domain</code>	String	Returns the domain name of the hosting server
<code>document.cookie</code>	String	Gets or sets HTTP cookies associated with the current document

Table 6.1: Essential Properties of the `document` Object.

The Danger of `document.write()`

While `document.write("text")` writes directly to the initial HTML parse stream, invoking `document.write()` **after the document has finished loading completely overwrites and destroys the entire existing webpage**. Avoid `document.write()` in modern web engineering.

6.4 DOM Element Selection Methods

JavaScript: DOM Element Selection in Practice

```
1 // 1. Traditional ID selection
2 const header = document.getElementById("main-header");
3
4 // 2. Traditional Class Name collection (HTMLCollection)
5 const activeItems = document.getElementsByClassName("nav-item");
6 console.log("First Nav Item:", activeItems[0]);
7
8 // 3. Modern CSS Selector (First match)
9 const primaryBtn = document.querySelector("button.btn-primary");
10
11 // 4. Modern All CSS Selectors (NodeList)
12 const allCards = document.querySelectorAll(".card-container
13   .card");
14 allCards.forEach((card, index) => {
15   console.log(`Card ${index + 1}:`, card);
16 });
```

6.5 DOM Elements: Creation, Deletion, and Tree Traversal

6.5.1 Creating and Appending Nodes

JavaScript: Dynamically Constructing and Inserting Elements

```
1 // Step 1: Create new <li> element node
2 const newListItem = document.createElement("li");
3
4 // Step 2: Set content and attributes
5 newListItem.textContent = "Unit 6: JavaScript DOM";
6 newListItem.className = "completed-module";
7
8 // Step 3: Locate target parent node
9 const moduleList = document.getElementById("courseModules");
10
11 // Step 4: Append child to parent
12 moduleList.appendChild(newListItem);
13
14 // Step 5: Removing a node
15 const obsoleteItem = document.getElementById("tempItem");
16 if (obsoleteItem) {
17   obsoleteItem.remove(); // Removes node directly from DOM
18 }
```

6.5.2 DOM Tree Traversal Properties

-
-

Method	Return Type	Matching Rule	Live Collection?
<code>getElementById(id)</code>	Single Element / null	Matches element by unique ID string	No
<code>getElementsByClassName(name)</code>	HTMLCollection	Matches all elements with class name	Yes** (Live)
<code>getElementsByTagName(tag)</code>	HTMLCollection	Matches all elements with tag name	Yes** (Live)
<code>querySelector(selector)</code>	Single Element / null	Matches first element satisfying CSS selector	Static
<code>querySelectorAll(selector)</code>	NodeList	Matches all elements satisfying CSS selector	Static** (Supports <code>.forEach()</code>)

Table 6.2: Comparison of Traditional vs. Modern DOM Selection Methods.

-
-

6.6 DOM HTML: Manipulating Content and Attributes

6.6.1 Content Manipulation: `innerHTML` vs. `textContent`

Property	HTML Parsing Behavior	Security & Performance
<code>innerHTML</code>	**Parses HTML tags** inside the assigned string, creating real DOM child nodes.	**Security Risk** : Susceptible to Cross-Site Scripting (XSS) attacks if rendering unsanitized user inputs.
<code>textContent</code>	Treats the assigned string as **pure literal text** (does NOT parse tags).	**Safe** : Immune to XSS injection; higher performance.
<code>innerText</code>	Renders text respecting computed CSS styling (ignores hidden elements).	Slower (triggers browser reflow calculation).

Table 6.3: Comparison of `innerHTML`, `textContent`, and `innerText`.

JavaScript: Content and Attribute Modification

```

1  const displayBox = document.getElementById("statusBox");
2
3  // textContent renders tags as literal characters:
4  // "<strong>Active</strong>"
5  displayBox.textContent = "<strong>Active</strong>";
6
7  // innerHTML parses tags into bold element
8  displayBox.innerHTML = "<strong>Active</strong>";
9
10 // Attribute Modification Methods
11 const userImage = document.getElementById("profilePic");
12 userImage.setAttribute("src", "avatar2.png");
13 console.log("Image source:", userImage.getAttribute("src"));
14 userImage.removeAttribute("alt");

```

6.7 DOM CSS: Dynamic Styling & The `classList` API

6.7.1 1. The `style` Property (Inline Styling)

JavaScript: Manipulating Inline Styles

```

1  const modal = document.getElementById("popupModal");
2
3  // CSS: background-color -> JS: backgroundColor
4  modal.style.backgroundColor = "#0f172a";

```

```
5 modal.style.fontSize = "18px";
6 modal.style.display = "block"; // Make visible
```

6.7.2 2. The classList API (Best Practice)

Method Call	Behavior	Return Value
<code>element.classList.add("active")</code>	Adds the specified class if absent	void
<code>element.classList.remove("active")</code>	Removes the class if present	void
<code>element.classList.toggle("active")</code>	Adds if absent; removes if present	Boolean
<code>element.classList.contains("active")</code>	Checks if class is present	Boolean (true/false)

Table 6.4: Methods of the DOMTokenList (classList) Interface.

JavaScript: ClassList State Machine Toggling

```
1 const themeBtn = document.getElementById("btnToggleTheme");
2
3 themeBtn.addEventListener("click", function() {
4     // Toggles 'dark-mode' class on the <body> tag
5     document.body.classList.toggle("dark-mode");
6
7     if (document.body.classList.contains("dark-mode")) {
8         themeBtn.innerText = "Switch to Light Mode";
9     } else {
10        themeBtn.innerText = "Switch to Dark Mode";
11    }
12 });
```

6.8 DOM Events, The Event Object & Event Propagation

6.8.1 The Event Object

-
-
-

6.8.2 Event Propagation: Bubbling vs. Capturing

- 1.
- 2.

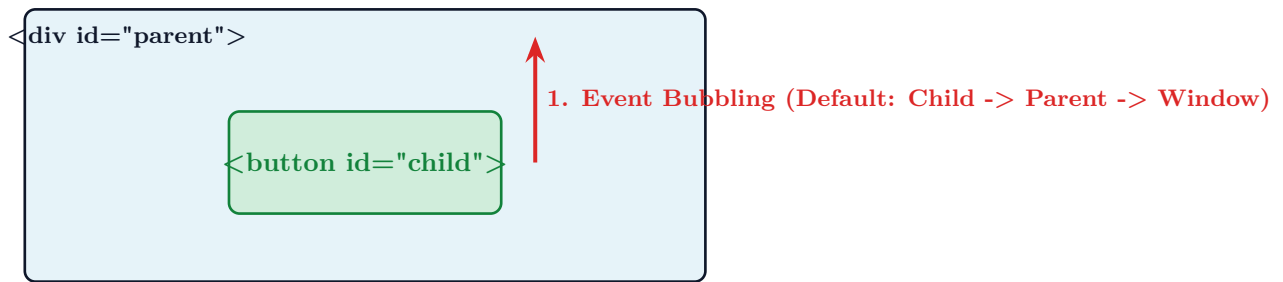


Figure 6.2: Event Bubbling Flow from Target Element Upwards to Ancestors.

6.9 Exam Tips & Common Student Pitfalls

Unit 6 High-Yield Traps

- **querySelector Prefix Requirement:** When selecting by class, `querySelector("active")` fails! You MUST include the period: `querySelector(".active")`.
- **NodeList vs HTMLCollection:** `querySelectorAll()` returns a static `NodeList` with `.forEach()`; `getElementsByClassName()` returns a live `HTMLCollection` lacking `.forEach()` on older engines.
- **CSS Property Names in JS:** Writing `el.style.background-color = "red"` causes a JavaScript `SyntaxError` (subtraction operator). Always use camelCase: `el.style.backgroundColor = "red"`.

6.10 Output Prediction & Debugging Challenges

Output Prediction: innerHTML vs textContent Output

Question: What will be rendered inside #box1 and #box2?

```

1 <div id="box1"></div>
2 <div id="box2"></div>
3 <script>
4   document.getElementById("box1").textContent = "<em>Hello</em>";
5   document.getElementById("box2").innerHTML = "<em>Hello</em>";
6 </script>

```

Answer & Layout Tracing:

- #box1 renders the literal characters: `<em>Hello</em>`
- #box2 parses the tag and renders in italics: *Hello*

Debugging Challenge: Fixing DOM Selector and Style Property Errors

Buggy Code:

```

1 const item = document.querySelector("my-card"); // Missing # or .

```

```
2 item.style.font-size = "20px"; // SyntaxError!
```

Diagnosis & Correction:

1. **Bug 1:** Selector string "my-card" looks for an element with tag name <my-card>. Add class dot ".my-card".
2. **Bug 2:** font-size must be written in camelCase fontSize.

Corrected Code:

```
1 const item = document.querySelector(".my-card");  
2 item.style.fontSize = "20px";
```

6.11 Graded Programming Practice Problems & Solutions

Unit 6 Practice Exercises

1. **Level 1 (Foundation):** Write JavaScript code to change the text content of a heading with id="title" to "Welcome to LPU".
2. **Level 1 (Foundation):** Select all paragraphs on a page and change their text color to dark blue using `querySelectorAll()` and `.forEach()`.
3. **Level 2 (Standard):** Write a function that creates a new button labeled "Delete", appends it to a card <div>, and adds a click event listener that removes the card from the DOM.
4. **Level 2 (Standard):** Implement a dark mode toggler using `classList.toggle("dark-theme")` and update the button label.
5. **Level 3 (Exam Tier):** Build an interactive shopping cart item counter with Increment (+) and Decrement (-) buttons updating a quantity label in the DOM.
6. **Level 3 (Exam Tier):** Create an image carousel script that changes the `src` attribute of an tag sequentially across an array of image URLs upon clicking a "Next" button.
7. **Level 4 (Challenge):** Design a complete interactive To-Do List application supporting adding items from an input field, marking items complete via class toggling, and deleting individual tasks.

6.11.1 Complete Step-by-Step Worked Solutions

Worked Solution: Solutions to Unit 6 Practice Problems

Solution to Problem 3 (Dynamic Card Removal):

```
1 const card = document.getElementById("studentCard");  
2 const delBtn = document.createElement("button");  
3 delBtn.textContent = "Delete Card";  
4 delBtn.className = "btn-danger";  
5
```

```
6 delBtn.addEventListener("click", function() {
7     card.remove(); // Removes parent card element completely
8 });
```

```
9
10 card.appendChild(delBtn);
```

Solution to Problem 7 (Interactive To-Do List):

```
1 const addBtn = document.getElementById("btnAdd");
2 const taskInput = document.getElementById("txtTask");
3 const taskList = document.getElementById("taskList");
4
5 addBtn.addEventListener("click", function() {
6     const text = taskInput.value.trim();
7     if (text === "") return;
8
9     // 1. Create list item
10    const li = document.createElement("li");
11    li.textContent = text;
12
13    // 2. Toggle completion on click
14    li.addEventListener("click", function() {
15        li.classList.toggle("completed");
16    });
17
18    // 3. Add delete action
19    const btnRemove = document.createElement("button");
20    btnRemove.textContent = " [X] ";
21    btnRemove.addEventListener("click", function(e) {
22        e.stopPropagation(); // Stop click from toggling completion
23        li.remove();
24    });
25
26    li.appendChild(btnRemove);
27    taskList.appendChild(li);
28    taskInput.value = ""; // Reset input
29 });
```

6.12 Multiple Choice Questions (MCQs)

1.
(a)
2.
(a)
3.
(a)

4.

(a)

5.

(a)

6.

(a)

7.

(a)

8.

(a)

9.

(a)

10.

(a)

6.13 Self-Assessment Unit Test

Unit 6 Examination (25 Marks — 45 Minutes)

Section A: Short Answer Concepts ($3 \times 2 = 6$ Marks)

1. Explain the difference between `innerHTML` and `textContent` with respect to security.
2. Differentiate between Event Bubbling and Event Capturing.
3. State two advantages of using `classList` over direct `style` property assignments.

Section B: Code Tracing & Analysis ($3 \times 3 = 9$ Marks)

4. Predict the output when the button is clicked twice:

```

1 <button id="btn">Click</button>
2 <script>
3     const b = document.getElementById("btn");
4     b.addEventListener("click", () => {
5         b.classList.toggle("active");
6         console.log(b.classList.contains("active"));
7     });
8 </script>
9

```

5. Explain how to dynamically create an image element and append it to `document.body`.

6. Differentiate between `querySelector()` and `querySelectorAll()`.

Section C: Comprehensive Programming Problem (1 × 10 = 10 Marks)

7. Write a complete JavaScript program that dynamically generates a Product Inventory Table:

- (a) Given an array of product objects `[id: 1, name: "Laptop", price: 50000, ...]`, dynamically create a `<table>` with headers and rows. (4 Marks)
- (b) Append an "Action" column containing a "Remove" button in each row. (3 Marks)
- (c) Add click event listeners to the remove buttons that delete the corresponding row from the table dynamically. (3 Marks)

6.13.1 Unit Test Complete Solutions & Rubric

Worked Solution: Complete Solutions for Unit 6 Test

Section A Solutions:

- innerHTML vs textContent Security** [2 Marks]: `innerHTML` parses HTML tags and executable scripts, making it vulnerable to XSS injection when displaying unsanitized user inputs. `textContent` treats all inputs as literal text, providing built-in injection immunity.
- Bubbling vs. Capturing** [2 Marks]: Bubbling propagates events upwards from the target child to ancestors. Capturing trickles events downwards from the window to the target child.
- classList Advantages** [2 Marks]: (1) Preserves clean separation of concerns by referencing pre-compiled CSS classes; (2) Enables easy multi-state toggling without overwriting other active classes.

Section B Solutions:

- Toggle Tracing** [3 Marks]: Click 1 adds "active" → logs `true`. Click 2 removes "active" → logs `false`.
- Dynamic Image** [3 Marks]:

```

1  const img = document.createElement("img");
2  img.src = "logo.png";
3  img.alt = "Portal Logo";
4  document.body.appendChild(img);

```

6. **QuerySelector vs All** [3 Marks]: `querySelector` returns only the first matching single Element; `querySelectorAll` returns a static `NodeList` containing all matching elements.

Section C Solution:

7. **Dynamic Inventory Table Implementation** [10 Marks]:

```

1  const products = [
2    { id: 101, name: "Mechanical Keyboard", price: 3500 },
3    { id: 102, name: "Gaming Mouse", price: 1800 },
4    { id: 103, name: "FHD Monitor", price: 12000 }
5  ];
6
7  function buildInventoryTable() {
8    const table = document.createElement("table");
9    table.border = "1";
10   table.style.width = "100%";
11   table.style.borderCollapse = "collapse";
12
13   // Header Row
14   const thead = document.createElement("thead");
15   thead.innerHTML = '<tr><th>ID</th><th>Product Name</th><th>Price (Rs.)</th><th>Action</th></tr>';
16   table.appendChild(thead);
17
18   // Body Rows
19   const tbody = document.createElement("tbody");
20   products.forEach(p => {
21     const tr = document.createElement("tr");
22
23     const tdId = document.createElement("td");
24     tdId.textContent = p.id;
25
26     const tdName = document.createElement("td");
27     tdName.textContent = p.name;
28
29     const tdPrice = document.createElement("td");
30     tdPrice.textContent = p.price;
31
32     const tdAction = document.createElement("td");
33     const btnDelete = document.createElement("button");
34     btnDelete.textContent = "Remove";
35     btnDelete.addEventListener("click", () => tr.remove());
36     tdAction.appendChild(btnDelete);
37
38     tr.append(tdId, tdName, tdPrice, tdAction);
39     tbody.appendChild(tr);
40   });
41
42   table.appendChild(tbody);

```

```
43      document.getElementById("inventoryContainer").appendChild(table);
44  }
45
46  buildInventoryTable();
```

Marking Scheme: Table & header generation: 4M; Row & data binding: 3M;
Dynamic row deletion event listener: 3M.

Master Web Cheatsheet & Top 50 Technical Viva-Voce Questions

7.1 Master HTML5 Syntax & Semantic Tag Matrix

7.2 HTML5 Form Controls & Multimedia Cheatsheet

7.3 CSS Selectors, Specificity & Box Model Matrix

7.4 JavaScript Operators, DOM Methods & Events Synthesis

7.5 Top 50 Technical Viva-Voce & Examination Questions

1.

2.

3.

4.

5.

6.

7.

HTML Tag	Type	Structural & Semantic Purpose	Key Attributes
<!DOCTYPE html>	DTD	Declares document type as standard HTML5 compliance mode	None
<html lang="en">	Container	Root container enclosing entire web document	lang
<head>	Container	Metadata container (non-visible configuration)	None
<meta>	Void	Encodes charset, viewport scaling, page descriptions	charset, name, content
<title>	Container	Defines browser tab title and search engine headline	None
<link>	Void	Links external CSS stylesheets and site favicons	rel, href, type
<body>	Container	Visible page content container	None
<h1>–<h6>	Block	Sectional headings in descending semantic hierarchy	None
<p>	Block	Paragraph flow of prose text (collapses whitespace)	None
 	Void	Inserts an inline line break	None
<hr>	Void	Inserts a thematic horizontal dividing rule	None
	Inline	Marks text of serious importance / urgency (semantic bold)	None
	Inline	Applies stress emphasis (semantic italic)	None
<mark>	Inline	Highlights text with yellow background notation	None
<small>	Inline	Side comments, copyright notices, and legal disclaimers	None
<abbr>	Inline	Acronym/abbreviation with tooltip expansion	title
<cite>	Inline	References the title of a creative work	None
<dfn>	Inline	Indicates defining instance of a technical term	None
<blockquote>	Block	Indented multi-line quotation block	cite (Source URL)
<q>	Inline	Inline quotation with automatic quotation marks	None
<!-- ... -->	Comment	Developer comments ignored by browser layout engine	None

Table 7.1: Comprehensive HTML5 Syntax and Tag Reference Matrix.

Form Element / Type	Tag Structure	Interaction & Data Processing Behavior	Required Form Encoding
<code><form></code>	Container	Defines submission endpoint and HTTP transfer protocol	<code>action</code> , <code>method</code> (GET/POST)
<code>type="text"</code>	<code><input></code>	Single-line alphanumeric text field	<code>name</code> , <code>placeholder</code> , <code>required</code>
<code>type="password"</code>	<code><input></code>	Character-masked password entry field	<code>name</code> , <code>required</code>
<code>type="email"</code>	<code><input></code>	Pattern-validated email entry field	<code>name</code> , <code>required</code>
<code>type="radio"</code>	<code><input></code>	Mutually exclusive single selection among shared <code>name</code>	Shared <code>name</code> , <code>value</code> , <code>checked</code>
<code>type="checkbox"</code>	<code><input></code>	Independent multi-selection toggling	<code>name</code> , <code>value</code> , <code>checked</code>
<code>type="file"</code>	<code><input></code>	File system upload selector	<code>enctype="multipart/form-data"</code>
<code><textarea></code>	Container	Multiline expandable text box	<code>rows</code> , <code>cols</code> , <code>name</code>
<code><select></code>	Container	Dropdown select menu populated by <code><option></code>	<code>name</code> , <code>multiple</code> , <code>selected</code>
<code>type="submit"</code>	<code><input></code>	Triggers form validation and transmission to server	<code>value</code>
<code>type="reset"</code>	<code><input></code>	Restores all form controls to default initial states	<code>value</code>
<code><audio></code>	Container	Native HTML5 audio playback player	<code>controls</code> , <code>autoplay</code> , <code>loop</code> , <code>muted</code>
<code><video></code>	Container	Native HTML5 video player with poster frame	<code>controls</code> , <code>width</code> , <code>height</code> , <code>poster</code>

Table 7.2: HTML5 Form Controls and Multimedia Cheatsheet.

Selector Category	Syntax Pattern	Specificity Weight	Matching Scope
Inline Style	<code>style="..."</code>	$(1, 0, 0, 0) = 1000$	Directly overrides internal/external styles
ID Selector	<code>#main-header</code>	$(0, 1, 0, 0) = 100$	Exactly one unique element per page
Class Selector	<code>.highlight</code>	$(0, 0, 1, 0) = 10$	All elements bearing class attribute
Pseudo-class	<code>:hover</code> , <code>:focus</code>	$(0, 0, 1, 0) = 10$	Dynamic user interaction states
Type (Element) Selector	<code>p</code> , <code>h1</code> , <code>div</code>	$(0, 0, 0, 1) = 1$	All elements matching the tag name
Universal Selector	<code>*</code>	$(0, 0, 0, 0) = 0$	Matches every element on the page

Table 7.3: CSS Selector Specificity Hierarchy and Weighting.

Box Model Property	Shorthand Syntax	Geometric Calculation Rule
<code>margin</code>	<code>margin: T R B L;</code>	Transparent space outside border. Auto margins (0 auto) center block elements.
<code>border</code>	<code>border: 2px solid #000;</code>	Visible boundary stroke around padding. Sets width, style, and color.
<code>padding</code>	<code>padding: 10px 20px;</code>	Inner transparent clearance between content and border.
<code>box-sizing: content-box</code>	Default model	Rendered Width = width + 2(padding) + 2(border)
<code>box-sizing: border-box</code>	Modern standard	Rendered Width = width (Padding and border absorbed within width)

Table 7.4: CSS Box Model Dimensions and Shorthand Reference.

DOM API Method	Return Value	Functional Description
<code>document.getElementById(id)</code>	Single Element / null	Fast lookup by unique ID string.
<code>document.getElementsByClassName(cls)</code>	HTMLCollection (Live)	Matches all elements by class name.
<code>document.querySelector(selector)</code>	Single Element / null	Returns first element matching CSS selector.
<code>document.querySelectorAll(selector)</code>	NodeList (Static)	Returns all matching elements; supports <code>.forEach()</code> .
<code>document.createElement(tagName)</code>	New Element Node	Instantiates a new unattached element node in memory.
<code>element.appendChild(childNode)</code>	Appended Node	Appends node as the last child of the parent element.
<code>element.remove()</code>	void	Removes element directly from the DOM tree.
<code>element.classList.add(cls)</code>	void	Adds CSS class to element's class collection.
<code>element.classList.toggle(cls)</code>	Boolean	Toggles CSS class presence dynamically.
<code>element.addEventListener(evt, fn)</code>	void	Attaches non-destructive event listener callback.

Table 7.5: Master JavaScript DOM Methods Reference.

8.

9.

10.

11.

12.

13.

14.

15.

16.

17.

18.

19.

20.

21.

22.

23.

24.

25.

26.

27.

28.

29.

30.

31.

32.

33.

34.

35.

36.

37.

38.

39.

40.

41.

42.

43.

44.

45.

46.

47.

48.

49.

50.

Comprehensive Practical Web Projects

8.1 Project 1: Developer Portfolio Profile Page

Project 1 Specification

Build a personal developer portfolio profile page using HTML5 semantic elements (<header>, <section>, <footer>), typography hierarchy, profile image with fallback, skills list, and CSS styling.

HTML: Developer Portfolio (index.html)

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width,
6          initial-scale=1.0">
7      <title>Aarav Sharma - Web Developer</title>
8      <link rel="stylesheet" href="portfolio.css">
9  </head>
10 <body>
11     <header class="profile-header">
12         
14         <h1>Aarav Sharma</h1>
15         <p class="tagline">Undergraduate Computer Science
16         Engineer | Web Developer</p>
17     </header>
18
19     <main class="container">
20         <section class="bio-section">
21             <h2>About Me</h2>
22             <p>
23                 Specializing in <strong>Full-Stack Web
24                 Development</strong>, focusing on
25                 standards-compliant <abbr title="HyperText Markup
26                 Language">HTML5</abbr>,
27                 <abbr title="Cascading Style Sheets">CSS3</abbr>,
28                 and modern JavaScript.
29             </p>
30         </section>
31
32         <section class="skills-section">
33             <h2>Core Competencies</h2>
34             <ul class="skills-list">

```

```

29         <li>Responsive Web Design & CSS Grid/Flexbox</li>
30         <li>Client-Side Form Validation & Regular
    Expressions</li>
31         <li>Dynamic DOM Tree Manipulation & Event
    Architecture</li>
32     </ul>
33 </section>
34 </main>
35
36 <footer class="profile-footer">
37     <p>&copy; 2026 Aarav Sharma. Built for Lovely
    Professional University.</p>
38 </footer>
39 </body>
40 </html>

```

CSS: Portfolio Stylesheet (portfolio.css)

```

1  body {
2      font-family: 'Segoe UI', Arial, sans-serif;
3      line-height: 1.6;
4      margin: 0;
5      padding: 0;
6      background-color: #f8fafc;
7      color: #1e293b;
8  }
9
10 .profile-header {
11     background-color: #0f172a;
12     color: #ffffff;
13     text-align: center;
14     padding: 40px 20px;
15 }
16
17 .avatar {
18     width: 120px;
19     height: 120px;
20     border-radius: 50%;
21     border: 4px solid #0284c7;
22 }
23
24 .tagline {
25     color: #94a3b8;
26     font-size: 16px;
27     margin-top: 5px;
28 }
29
30 .container {
31     max-width: 800px;
32     margin: 30px auto;
33     padding: 0 20px;
34 }
35
36 .skills-list li {
37     background: #ffffff;

```

```

38     margin: 8px 0;
39     padding: 12px 16px;
40     border-left: 4px solid #0284c7;
41     border-radius: 4px;
42     box-shadow: 0 1px 3px rgba(0,0,0,0.05);
43 }
44
45 .profile-footer {
46     text-align: center;
47     padding: 20px;
48     font-size: 14px;
49     color: #64748b;
50     border-top: 1px solid #e2e8f0;
51 }

```

8.2 Project 2: Academic Timetable & Course Portal

Project 2 Specification

Design an academic lecture schedule utilizing complex table structures, colspan lunch breaks, rowspan laboratory slots, and CSS hover highlights.

HTML: Academic Timetable (timetable.html)

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>CSE Semester Schedule</title>
6      <style>
7          table {
8              width: 100%;
9              border-collapse: collapse;
10             font-family: Arial, sans-serif;
11             margin-top: 20px;
12         }
13         th, td {
14             border: 1px solid #cbd5e1;
15             padding: 12px;
16             text-align: center;
17         }
18         th {
19             background-color: #0284c7;
20             color: #ffffff;
21         }
22         tr:nth-child(even) { background-color: #f8fafc; }
23         tr:hover { background-color: #e0f2fe; }
24         .break { background-color: #fee2e2; font-weight: bold;
25             color: #991b1b; }
26     </style>
27 </head>
28 <body>

```

```

28     <h2>B.Tech CSE - Semester 1 Schedule</h2>
29     <table>
30         <thead>
31             <tr>
32                 <th>Day</th>
33                 <th>09:00 - 10:00</th>
34                 <th>10:00 - 11:00</th>
35                 <th>11:00 - 12:00</th>
36                 <th>12:00 - 01:00</th>
37                 <th>01:00 - 03:00</th>
38             </tr>
39         </thead>
40         <tbody>
41             <tr>
42                 <td>Monday</td>
43                 <td>CSE326 (Web)</td>
44                 <td>MTH165 (Math)</td>
45                 <td>PHY101 (Physics)</td>
46                 <td rowspan="5" class="break">LUNCH BREAK</td>
47                 <td>INT108 Lab (Python)</td>
48             </tr>
49             <tr>
50                 <td>Tuesday</td>
51                 <td>CSE111 (IT)</td>
52                 <td>CSE326 (Web)</td>
53                 <td>MTH165 (Math)</td>
54                 <td>PHY101 Lab (Physics)</td>
55             </tr>
56             <tr>
57                 <td>Wednesday</td>
58                 <td>INT108 (Python)</td>
59                 <td>PHY101 (Physics)</td>
60                 <td>CSE326 (Web)</td>
61                 <td>CSE326 Web Dev Lab</td>
62             </tr>
63         </tbody>
64     </table>
65 </body>
66 </html>

```

8.3 Project 3: Interactive Semester CGPA Calculator

Project 3 Specification

Implement an interactive CGPA calculator using JavaScript functions, array parsing, grade-point mapping, and dynamic DOM output updates.

HTML: CGPA Calculator (cgpa.html)

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>

```

```

4     <meta charset="UTF-8">
5     <title>CGPA Calculator</title>
6     <style>
7         .calc-card {
8             max-width: 400px;
9             margin: 40px auto;
10            padding: 25px;
11            border-radius: 8px;
12            background: #ffffff;
13            box-shadow: 0 4px 12px rgba(0,0,0,0.1);
14            font-family: Arial, sans-serif;
15        }
16        .form-group { margin-bottom: 15px; }
17        label { display: block; font-weight: bold; margin-bottom:
18    5px; }
19        input { width: 100%; padding: 8px; box-sizing:
20    border-box; }
21        button { width: 100%; padding: 10px; background: #16a34a;
22    color: white; border: none; font-weight: bold; cursor:
23    pointer; }
24        .result-box { margin-top: 20px; font-size: 18px;
25    font-weight: bold; text-align: center; color: #0f172a; }
26    </style>
27    </head>
28    <body>
29        <div class="calc-card">
30            <h2>Semester CGPA Calculator</h2>
31            <div class="form-group">
32                <label>CSE326 (Web Dev - 3 Cr):</label>
33                <input type="number" id="g1" placeholder="Grade Point
34    (0-10)">
35            </div>
36            <div class="form-group">
37                <label>MTH165 (Math - 4 Cr):</label>
38                <input type="number" id="g2" placeholder="Grade Point
39    (0-10)">
40            </div>
41            <div class="form-group">
42                <label>INT108 (Python - 3 Cr):</label>
43                <input type="number" id="g3" placeholder="Grade Point
44    (0-10)">
45            </div>
46            <button id="btnCompute">Calculate CGPA</button>
47            <div class="result-box" id="displayCGPA"></div>
48        </div>
49
50        <script>
51
52    document.getElementById("btnCompute").addEventListener("click",
53    function() {
54        const v1 =
55    parseFloat(document.getElementById("g1").value) || 0;
56        const v2 =
57    parseFloat(document.getElementById("g2").value) || 0;
58        const v3 =
59    parseFloat(document.getElementById("g3").value) || 0;

```

```

47
48         const totalPoints = (v1 * 3) + (v2 * 4) + (v3 * 3);
49         const totalCredits = 3 + 4 + 3; // 10 Credits
50         const cgpa = totalPoints / totalCredits;
51
52         document.getElementById("displayCGPA").innerText =
53         'Calculated CGPA: ${cgpa.toFixed(2)} / 10.0';
54     });
55 </script>
56 </body>
57 </html>

```

8.4 Project 4: Comprehensive Registration Form Validator

Project 4 Specification

Construct an industrial-grade form validation engine checking name lengths, email regex patterns, password confirmation matches, and phone number structures.

HTML: Form Validation Engine (validator.html)

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Registration Validator</title>
6     <style>
7         .form-container { max-width: 450px; margin: 30px auto;
8         padding: 25px; border: 1px solid #cbd5e1; border-radius: 8px;
9         font-family: Arial; }
10        .input-control { width: 100%; padding: 10px; margin: 6px
11        0 16px; border: 1px solid #94a3b8; border-radius: 4px;
12        box-sizing: border-box; }
13        .error-msg { color: #dc2626; font-size: 13px;
14        font-weight: bold; margin-bottom: 12px; }
15        .btn-submit { width: 100%; padding: 12px; background:
16        #0284c7; color: white; border: none; border-radius: 4px;
17        font-weight: bold; cursor: pointer; }
18    </style>
19 </head>
20 <body>
21     <div class="form-container">
22         <h2>University Registration</h2>
23         <form id="regForm">
24             <label for="txtName">Full Name:</label>
25             <input type="text" id="txtName" class="input-control">
26
27             <label for="txtEmail">Email Address:</label>
28             <input type="text" id="txtEmail"
29             class="input-control">
30
31             <label for="txtPhone">Mobile Number (10

```

```
digits):</label>
24     <input type="text" id="txtPhone"
    class="input-control">
25
26     <label for="txtPass">Password (Min 8 chars):</label>
27     <input type="password" id="txtPass"
    class="input-control">
28
29     <div id="errorAlert" class="error-msg"></div>
30     <button type="submit" class="btn-submit">Submit
Application</button>
31   </form>
32 </div>
33
34   <script>
35
    document.getElementById("regForm").addEventListener("submit",
    function(e) {
36        const name =
    document.getElementById("txtName").value.trim();
37        const email =
    document.getElementById("txtEmail").value.trim();
38        const phone =
    document.getElementById("txtPhone").value.trim();
39        const pass = document.getElementById("txtPass").value;
40        const errBox = document.getElementById("errorAlert");
41
42        let err = "";
43
44        if (name.length < 3) {
45            err = "Name must be at least 3 characters.";
46        } else if (!/^[\s@]+@[\s@]+\.[\s@]+$/ .test(email))
    {
47            err = "Please enter a valid email address.";
48        } else if (!/^d{10}$/.test(phone)) {
49            err = "Mobile number must be exactly 10 digits.";
50        } else if (pass.length < 8) {
51            err = "Password must be at least 8 characters.";
52        }
53
54        if (err !== "") {
55            e.preventDefault();
56            errBox.innerText = err;
57        } else {
58            errBox.innerText = "";
59            alert("Form submitted successfully!");
60        }
61    });
62   </script>
63 </body>
64 </html>
```

8.5 Project 5: Interactive Dynamic To-Do List Application

Project 5 Specification

Design an interactive To-Do application utilizing dynamic node creation (`createElement`), node removal (`remove`), event delegation, and `classList` toggling.

HTML: Interactive To-Do List (todo.html)

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Dynamic Task Manager</title>
6      <style>
7          .todo-card { max-width: 400px; margin: 40px auto;
padding: 25px; border-radius: 8px; background: white;
box-shadow: 0 4px 12px rgba(0,0,0,0.1); font-family: Arial; }
8          .input-row { display: flex; gap: 8px; margin-bottom:
15px; }
9          .input-row input { flex: 1; padding: 8px; }
10         .input-row button { padding: 8px 16px; background:
#0284c7; color: white; border: none; cursor: pointer; }
11         ul { list-style: none; padding: 0; }
12         li { display: flex; justify-content: space-between;
padding: 10px; border-bottom: 1px solid #e2e8f0; cursor:
pointer; }
13         li.done { text-decoration: line-through; color: #94a3b8; }
14         .btn-del { color: #dc2626; border: none; background:
none; font-weight: bold; cursor: pointer; }
15     </style>
16 </head>
17 <body>
18     <div class="todo-card">
19         <h2>CSE326 Task Manager</h2>
20         <div class="input-row">
21             <input type="text" id="taskInput" placeholder="Add
new learning task...">
22             <button id="btnAddTask">Add</button>
23         </div>
24         <ul id="taskList"></ul>
25     </div>
26
27     <script>
28         const input = document.getElementById("taskInput");
29         const addBtn = document.getElementById("btnAddTask");
30         const list = document.getElementById("taskList");
31
32         addBtn.addEventListener("click", function() {
33             const taskText = input.value.trim();
34             if (taskText === "") return;
35
36             const li = document.createElement("li");
37             li.textContent = taskText;
38
39             // Toggle Done State

```

```

40         li.addEventListener("click", function() {
41             li.classList.toggle("done");
42         });
43
44         // Delete Button
45         const delBtn = document.createElement("button");
46         delBtn.textContent = "X";
47         delBtn.className = "btn-del";
48         delBtn.addEventListener("click", function(e) {
49             e.stopPropagation(); // Prevent toggling done
50
51             class
52                 li.remove();
53             });
54
55             li.appendChild(delBtn);
56             list.appendChild(li);
57             input.value = "";
58         });
59     </script>
60 </body>
61 </html>

```

8.6 Project 6: Real-time Filterable Student Directory

Project 6 Specification

Build a real-time filterable student directory where typing into an input field instantly filters a table of student records using JavaScript input events and `textContent.toLowerCase().includes()`.

HTML: Filterable Directory (search.html)

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Student Search Directory</title>
6     <style>
7         .search-container { max-width: 600px; margin: 30px auto;
8         font-family: Arial; }
9         .search-box { width: 100%; padding: 10px; margin-bottom:
10        15px; border: 1px solid #94a3b8; border-radius: 4px;
11        box-sizing: border-box; }
12        table { width: 100%; border-collapse: collapse; }
13        th, td { border: 1px solid #cbd5e1; padding: 10px;
14        text-align: left; }
15        th { background: #0284c7; color: white; }
16        tr:nth-child(even) { background: #f8fafc; }
17        .no-match { display: none; }
18    </style>
19 </head>
20 <body>

```

```

17     <div class="search-container">
18         <h2>CSE Student Search Directory</h2>
19         <input type="text" id="txtSearch" class="search-box"
placeholder="Search by name or department...">
20
21         <table id="studentTable">
22             <thead>
23                 <tr><th>Registration ID</th><th>Student
Name</th><th>Department</th></tr>
24             </thead>
25             <tbody>
26                 <tr><td>12601001</td><td>Aarav
Sharma</td><td>Computer Science</td></tr>
27                 <tr><td>12601002</td><td>Bhavna
Patel</td><td>Information Technology</td></tr>
28                 <tr><td>12601003</td><td>Chirag
Verma</td><td>Computer Science</td></tr>
29                 <tr><td>12601004</td><td>Divya Reddy</td><td>AI &
Data Science</td></tr>
30             </tbody>
31         </table>
32     </div>
33
34     <script>
35         const searchInput = document.getElementById("txtSearch");
36         const tableRows =
document.querySelectorAll("#studentTable tbody tr");
37
38         searchInput.addEventListener("input", function() {
39             const query = searchInput.value.toLowerCase();
40
41             tableRows.forEach(row => {
42                 const text = row.textContent.toLowerCase();
43                 if (text.includes(query)) {
44                     row.style.display = ""; // Show row
45                 } else {
46                     row.style.display = "none"; // Hide row
47                 }
48             });
49         });
50     </script>
51 </body>
52 </html>

```

8.7 Project 7: Interactive Accordion FAQ Component

Project 7 Specification

Construct an accessible FAQ accordion component that expands and collapses content panels upon clicking question headers using `classList.toggle()` and CSS transitions.

HTML: Accordion Component (accordion.html)

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Course FAQ Accordion</title>
6   <style>
7     .accordion-container { max-width: 550px; margin: 30px
8 auto; font-family: Arial; }
9     .accordion-item { border: 1px solid #cbd5e1;
10 margin-bottom: 8px; border-radius: 4px; overflow: hidden; }
11     .accordion-header { background: #0f172a; color: white;
12 padding: 12px 16px; cursor: pointer; font-weight: bold; }
13     .accordion-content { max-height: 0; overflow: hidden;
14 transition: max-height 0.3s ease, padding 0.3s ease;
15 background: #ffffff; padding: 0 16px; }
16     .accordion-item.active .accordion-content { max-height:
17 150px; padding: 12px 16px; }
18   </style>
19 </head>
20 <body>
21   <div class="accordion-container">
22     <h2>Frequently Asked Questions</h2>
23     <div class="accordion-item">
24       <div class="accordion-header">What are the
25 prerequisites for CSE326?</div>
26       <div class="accordion-content">Basic algorithmic
27 logic and fundamental programming syntax.</div>
28     </div>
29     <div class="accordion-item">
30       <div class="accordion-header">Is JavaScript required
31 for the final examination?</div>
32       <div class="accordion-content">Yes, Units 4, 5, and 6
33 heavily focus on JavaScript syntax, events, and DOM
34 manipulation.</div>
35     </div>
36   </div>
37   <script>
38     const headers =
39 document.querySelectorAll(".accordion-header");
40
41     headers.forEach(header => {
42       header.addEventListener("click", function() {
43         const item = header.parentElement;
44         item.classList.toggle("active");
45       });
46     });
47   </script>
48 </body>
49 </html>
```

8.8 Project 8: Interactive Lightbox Image Gallery

Project 8 Specification

Design an interactive image thumbnail gallery where clicking any thumbnail displays a high-resolution modal overlay with animated backdrop and close button.

HTML: Modal Image Gallery (lightbox.html)

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Campus Gallery</title>
6      <style>
7          .gallery { display: flex; gap: 15px; justify-content:
8              center; margin-top: 40px; }
9          .thumbnail { width: 120px; height: 80px; object-fit:
10             cover; cursor: pointer; border-radius: 4px; border: 2px solid
11             #0284c7; }
12          .lightbox { display: none; position: fixed; top: 0; left:
13             0; width: 100%; height: 100%; background: rgba(0,0,0,0.8);
14             justify-content: center; align-items: center; }
15          .lightbox img { max-width: 80%; max-height: 80%;
16             border-radius: 6px; }
17          .close-btn { position: absolute; top: 20px; right: 30px;
18             color: white; font-size: 30px; cursor: pointer; }
19      </style>
20 </head>
21 <body>
22     <div class="gallery">
23         
25         
27     </div>
28
29     <div class="lightbox" id="modalBox">
30         <span class="close-btn" id="btnClose">&times;</span>
31         <img id="modalImg" src="" alt="Full view">
32     </div>
33
34     <script>
35         const modal = document.getElementById("modalBox");
36         const modalImg = document.getElementById("modalImg");
37         const thumbs = document.querySelectorAll(".thumbnail");
38         const btnClose = document.getElementById("btnClose");
39
40         thumbs.forEach(thumb => {
41             thumb.addEventListener("click", () => {
42                 modalImg.src = thumb.getAttribute("data-full");
43                 modal.style.display = "flex";
44             });
45         });
46
47         btnClose.addEventListener("click", () => {

```

```
        modal.style.display = "none"; });  
39         modal.addEventListener("click", (e) => { if (e.target ===  
        modal) modal.style.display = "none"; });  
40     </script>  
41 </body>  
42 </html>
```

Full-Length University Mock Examinations

9.1 University Mock Examination 1

CSE326 Mock Examination 1 (100 Marks — 3 Hours)

Section A: Short Compulsory Questions ($10 \times 2 = 20$ Marks)

1. Explain the role of the `<!DOCTYPE html>` declaration.
2. What is the difference between `<strong>` and `<b>`?
3. Give two examples of unpaired (void) HTML tags.
4. How do you open an anchor link in a new browser tab?
5. What is the purpose of `border-collapse: collapse;` in CSS table styling?
6. Differentiate between `box-sizing: content-box` and `border-box`.
7. Explain the difference between `var` and `let` in variable scoping.
8. What does `5 === "5"` evaluate to and why?
9. State the difference between `innerHTML` and `textContent`.
10. What is the purpose of `event.preventDefault()` in form validation?

Section B: Analytical Medium Problems ($6 \times 5 = 30$ Marks)

11. Explain the difference between Block-level and Inline elements with two HTML examples of each.
12. Differentiate between HTTP GET and POST methods during form submissions.
13. Explain the four concentric layers of the CSS Box Model with an annotated diagram.
14. Predict the output of the following JavaScript code:

```
1 console.log("10" + 5, "10" - 5);
2 console.log(0 == false, 0 === false);
3
```
15. Explain how the `arguments` object works in JavaScript functions with a code example.
16. Differentiate between Event Bubbling and Event Capturing in the DOM.

Section C: Comprehensive Programming Problems (5 × 10 = 50 Marks)

17. Write a complete, standards-compliant HTML5 document for a University Faculty Directory containing metadata, semantic headers, an address using `<br>`, and formatted contact details. (10 Marks)
18. Construct an academic timetable table in HTML using `colspan` for lunch breaks, `rowspan` for laboratory slots, and style it with alternating row colors and hover highlights using CSS. (10 Marks)
19. Design a complete CSS stylesheet for a modern Product Showcase Card, featuring box-sizing, padding, borders, shadows, and interactive hover elevation animations. (10 Marks)
20. Write a complete JavaScript form validation script for an admission form checking non-empty student names, 10-digit mobile numbers via regex, and email structure, preventing submission upon errors. (10 Marks)
21. Implement a dynamic To-Do List web application using JavaScript DOM methods (`createElement`, `appendChild`, `remove`, `classList.toggle`) that adds and deletes tasks dynamically. (10 Marks)

9.1.1 Complete Worked Solutions for Mock Examination 1**Worked Solution: Solutions to Mock Exam 1 (100 Marks)****Section A Solutions (10 × 2 = 20 Marks):**

1. Instructs the browser to render the page in standard HTML5 mode rather than quirks mode. [2M]
2. `<b>` is visual only; `<strong>` conveys semantic importance for screen readers and SEO. [2M]
3. `<br>` and `<img>`. [2M]
4. `<a href="url" target="_blank">`. [2M]
5. Merges double borders of adjacent table cells into a single unified border line. [2M]
6. In `content-box`, padding/border add to width; in `border-box`, padding/border are absorbed within width. [2M]
7. `var` is function-scoped; `let` is block-scoped (`{}`). [2M]
8. `false` because `===` strictly evaluates both value and data type without coercion (Number vs String). [2M]
9. `innerHTML` parses HTML tags (XSS risk); `textContent` inserts safe literal text. [2M]
10. Halts default browser form submission when client-side validation fails. [2M]

Section B Solutions (6 × 5 = 30 Marks):

11. Block elements (`<div>`, `<p>`) start on a new line and expand to 100% width. Inline elements (`<span>`, `<a>`) flow with text and only occupy content width. [5M]

12. GET appends data to URL query strings (insecure, length-limited); POST transmits payloads inside HTTP request body (secure, unlimited length, supports file uploads). [5M]
13. Box Model: Content Box (innermost) → Padding → Border → Margin (outermost). [5M]
14. Outputs: "105" 5 and true false. [5M]
15. arguments is an array-like object containing all parameters passed during invocation. Example: `function sum() { let s=0; for(let v of arguments) s+=v; return s; }`. [5M]
16. Bubbling triggers events from the target element upwards to ancestors; Capturing trickles events downwards from window to target. [5M]

Section C Solutions (5 × 10 = 50 Marks):

17. **Faculty Directory HTML5 Document:** Includes DOCTYPE, UTF-8, viewport, `<h1>`, `<h2>`, `<p>`, `<abbr>`, `<strong>`, `<br>`. [10M]
18. **Academic Timetable Table:** Implements `<table>` with `border-collapse: collapse;`, `colspan="5"` for lunch break, `rowspan="2"` for lab, and CSS `tr:hover { background-color: #e0f2fe; }`. [10M]
19. **Product Showcase Card CSS:** Sets `width: 300px;` `box-sizing: border-box;` `padding: 20px;` `border-radius: 8px;` `box-shadow: 0 4px 6px rgba(0,0,0,0.1);` `transition: transform 0.2s;` and `:hover { transform: translateY(-5px); }`. [10M]
20. **Form Validation Engine:** Intercepts submit, validates non-empty name, regex `/^\d{10}$/` for phone, regex `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` for email, and halts with `e.preventDefault()`. [10M]
21. **Dynamic To-Do List Application:** Implements `document.createElement("li")`, attaches click listener toggling `classList.toggle("completed")`, adds delete button with `li.remove()`, and appends to parent list. [10M]

9.2 University Mock Examination 2

CSE326 Mock Examination 2 (100 Marks — 3 Hours)

Section A: Short Compulsory Questions (10 × 2 = 20 Marks)

1. What is the purpose of the viewport meta tag?
2. How does `<del>` differ from `<small>`?
3. State the difference between `type="radio"` and `type="checkbox"`.
4. What attribute is required on `<form>` for uploading files?
5. How do you apply inline CSS to an element?

6. What is the difference between `background-size: cover` and `contain`?
7. What does `confirm()` return when the user clicks Cancel?
8. What is the result of `typeof null` in JavaScript?
9. Which DOM method returns a live `HTMLCollection`?
10. How do you toggle a CSS class using the `classList` API?

Section B: Analytical Medium Problems ($6 \times 5 = 30$ Marks)

11. Explain the semantic meaning of `<abbr>`, `<cite>`, and `<dfn>` with code snippets.
12. Write HTML code creating an ordered list numbered with uppercase Roman numerals starting from IV.
13. Explain CSS selector specificity and order the following: `p`, `#title`, `.highlight`, `style="..."`.
14. Write a JavaScript object representing a `student` with properties and a `getSummary()` method using `this`.
15. Differentiate between `while` and `do...while` loops in JavaScript with code examples.
16. Explain how to dynamically create, configure, and append an image node to the DOM.

Section C: Comprehensive Programming Problems ($5 \times 10 = 50$ Marks)

17. Design a complete University Course Registration Form featuring text fields, radio buttons with shared name, elective checkboxes, a dropdown select menu, and file upload. (10 Marks)
18. Build an embedded multimedia learning page containing native `<audio>` and `<video>` players with poster frames, fallback formats, and playback controls. (10 Marks)
19. Write a complete external CSS stylesheet for an interactive data table with sticky headers, zebra striping, and cell padding. (10 Marks)
20. Write a JavaScript password validation script that checks min 8 characters, at least one digit, and matching password confirmation. (10 Marks)
21. Implement a dynamic Shopping Cart summary in JavaScript that calculates subtotal, tax (18%), and updates DOM text nodes upon clicking an "Apply Discount" button. (10 Marks)

9.2.1 Complete Worked Solutions for Mock Examination 2

Worked Solution: Solutions to Mock Exam 2 (100 Marks)

Section A Solutions ($10 \times 2 = 20$ Marks):

1. Configures responsive mobile scaling to match physical device screen width with a 1:1 initial zoom. [2M]
2. `<del>` renders strikethrough text for deleted content; `<small>` renders smaller text

- for legal side notes. [2M]
- 3. Radio buttons allow selecting only one option from a group; checkboxes allow selecting multiple options independently. [2M]
- 4. `enctype="multipart/form-data"` with `method="POST"`. [2M]
- 5. Using the `style` attribute: `<p style="color: blue;">`. [2M]
- 6. `cover` scales image to fill the container completely; `contain` scales image to fit within container boundaries without clipping. [2M]
- 7. Boolean `false`. [2M]
- 8. `"object"`. [2M]
- 9. `document.getElementsByClassName()` or `document.getElementsByTagName()`. [2M]
- 10. `element.classList.toggle("className")`. [2M]

Section B Solutions (6 × 5 = 30 Marks):

- 11. `<abbr title="desc">` defines abbreviations; `<cite>` titles of creative works; `<dfn>` defining instance of terms. [5M]
- 12. `<ol type="I" start="4"> <li>Item 1</li> <li>Item 2</li> </ol>`. [5M]
- 13. Specificity order: `style="..."` (1000) > `#title` (100) > `.highlight` (10) > `p` (1). [5M]
- 14. Student Object: `const s = { name: "Aarav", gpa: 9.0, getSummary: function() { return `${this.name}: ${this.gpa}`; } };` [5M]
- 15. `while` checks condition pre-test (0 or more executions); `do...while` checks condition post-test (at least 1 execution). [5M]
- 16. `const img = document.createElement("img"); img.src="pic.jpg"; img.alt="Pic"; document.body.appendChild(img);`. [5M]

Section C Solutions (5 × 10 = 50 Marks):

- 17. **Registration Form:** Implements `<form method="POST" enctype="multipart/form-data">`, text/email inputs, radio gender group, checkboxes, `<select>`, and file input. [10M]
- 18. **Multimedia Webpage:** Constructs `<video controls poster="thumb.jpg"> <source src="v.mp4" type="video/mp4"> </video>` and `<audio controls>`. [10M]
- 19. **Table Stylesheet:** Implements `border-collapse: collapse;`, `th { background: #0284c7; color: white; }, tr:nth-child(even) { background: #f8fafc; },` and `tr:hover`. [10M]
- 20. **Password Validator:** Validates `length ≥ 8`, regex `/\d/`, and equality check `p1 === p2`, reporting errors in DOM. [10M]
- 21. **Shopping Cart Engine:** Computes items total, adds 18% tax, applies 10% discount on button click, and updates `textContent` of summary elements. [10M]

9.3 University Mock Examination 3

CSE326 Mock Examination 3 (100 Marks — 3 Hours)

Section A: Short Compulsory Questions ($10 \times 2 = 20$ Marks)

1. Explain the meaning and function of the `<dfn>` tag.
2. How does `<del>` differ from `<s>`?
3. What is the purpose of the `charset="UTF-8"` metadata tag?
4. How do you configure a video element to loop continuously?
5. What is the default `text-align` property value in CSS?
6. Explain the function of `font-family` font stacks.
7. What is the difference between `break` and `continue` in JavaScript loops?
8. What does `isNaN("CSE326")` return?
9. How do you remove a child element from the DOM?
10. What does `event.target` refer to in event handling?

Section B: Analytical Medium Problems ($6 \times 5 = 30$ Marks)

11. Write HTML code creating a definition list describing three web development terms.
12. Explain the CSS Box Model calculation for total rendered element width under default box-sizing.
13. Predict the output of this JavaScript snippet:

```
1 function calc() {  
2     let a = 5;  
3     if (true) {  
4         let a = 10;  
5         var b = 20;  
6     }  
7     console.log(a, b);  
8 }  
9 calc();  
10
```

14. Write a function that accepts arbitrary numbers using `arguments` and returns the maximum value.
15. Differentiate between `innerHTML`, `innerText`, and `textContent`.
16. Explain the difference between `inline`, `DOM` property, and `addEventListener` event handling.

Section C: Comprehensive Programming Problems ($5 \times 10 = 50$ Marks)

17. Design a complete University Course Syllabus page utilizing semantic HTML5 tags (<header>, <section>, <article>, <footer>), nested lists, and metadata. (10 Marks)
18. Construct a multi-column pricing table using HTML tables, colspan, rowspan, and modern CSS styling with hover highlights. (10 Marks)
19. Write a complete CSS stylesheet for a horizontal navigation bar with drop-down menus using hover pseudo-classes. (10 Marks)
20. Write a complete JavaScript form validation script checking student name length, email regex, 10-digit mobile number, and age bounds (18 to 35). (10 Marks)
21. Build an interactive image carousel in JavaScript that advances images sequentially upon clicking a "Next" button and cycles back to the start. (10 Marks)

9.3.1 Complete Worked Solutions for Mock Examination 3

Worked Solution: Solutions to Mock Exam 3 (100 Marks)

Section A Solutions (10 × 2 = 20 Marks):

1. Indicates the defining instance of a term being defined in the document. [2M]
2. conveys semantic deletion with datetime tracking; <s> represents visual strikethrough for inaccurate data. [2M]
3. Enables global character encoding supporting international alphabets and symbols. [2M]
4. Adding the boolean attribute loop to the <video> tag. [2M]
5. left (or right for right-to-left languages). [2M]
6. Provides a prioritized fallback list of font typefaces if the primary font is missing on the client OS. [2M]
7. break terminates the loop immediately; continue skips the current iteration and moves to the next. [2M]
8. true (cannot be parsed to a number). [2M]
9. parentElement.removeChild(childElement) or childElement.remove(). [2M]
10. Points to the specific innermost element that dispatched the event. [2M]

Section B Solutions (6 × 5 = 30 Marks):

11. Definition list: <dl> <dt>HTML</dt><dd>Markup</dd>
 <dt>CSS</dt><dd>Style</dd> <dt>JS</dt><dd>Logic</dd> </dl>. [5M]
12. Total Width = width + padding-left + padding-right + border-left +
 border-right. [5M]
13. Output: 5 20 (inner let a is shadowed within block; var b is function-scoped). [5M]
14. Max Function: function findMax() { let m = -Infinity; for(let v of
 arguments) if(v>m) m=v; return m; }. [5M]

15. `innerHTML` parses tags; `textContent` returns literal text; `innerText` returns rendered text respecting CSS display. [5M]
16. Inline mixes HTML/JS; DOM property allows only one function; `addEventListener` supports multiple independent listeners. [5M]

Section C Solutions (5 × 10 = 50 Marks):

17. **Syllabus Webpage:** Fully semantic document structure with `<header>`, `<main>`, `<section>`, nested lists, and metadata. [10M]
18. **Pricing Table:** Implements `<table>` with `border-collapse`, highlighted "Recommended" tier via CSS, and `colspan="3"`. [10M]
19. **Horizontal Navbar CSS:** Styles `<ul>` with `display: flex; list-style: none; a { text-decoration: none; padding: 12px; }` and `a:hover`. [10M]
20. **Comprehensive Form Validation:** Validates name length, email regex `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`, phone `/^\d{10}$/`, and age `18 ≤ age ≤ 35`. [10M]
21. **Interactive Image Carousel:** Array of images, current index pointer, `btnNext.addEventListener("click", () => { idx = (idx + 1) % imgs.length; img.src = imgs[idx]; })`. [10M]

9.4 University Mock Examination 4

CSE326 Mock Examination 4 (100 Marks — 3 Hours)

Section A: Short Compulsory Questions (10 × 2 = 20 Marks)

1. State two advantages of HTML5 semantic tags over generic `<div>` elements.
2. What is the difference between `<q>` and `<blockquote>`?
3. What does `<input type="reset">` do?
4. Which CSS selector targets all paragraphs inside a `<div class="content">`?
5. What does `text-transform: capitalize` accomplish?
6. What is the difference between `null` and `undefined` in JavaScript?
7. What is the Temporal Dead Zone (TDZ)?
8. How do you access an object property dynamically using a variable?
9. What is the purpose of `classList.toggle()`?
10. Explain the term event delegation.

Section B: Analytical Medium Problems (6 × 5 = 30 Marks)

11. Construct an HTML audio player supporting MP3 and OGG formats with playback controls.

12. Explain CSS selector specificity hierarchy with numerical weights.

13. Predict the output of this JavaScript object method program:

```
1  const user = {
2      name: "Kavya",
3      greet: function() { return 'Hello, ${this.name}'; }
4  };
5  console.log(user.greet());
6
```

14. Write a function that checks if a string is a palindrome in JavaScript.

15. Explain how `document.querySelectorAll()` differs from `document.getElementsByTagName()`.

16. Differentiate between `focus` and `blur` events in web forms.

Section C: Comprehensive Programming Problems (5 × 10 = 50 Marks)

17. Design a complete University Feedback Portal form containing rating radio groups, interest checkboxes, a feedback textarea, and file upload. (10 Marks)

18. Build an interactive Product Card with responsive image, title, price, discount badge, and hover elevation animation using CSS. (10 Marks)

19. Write a complete JavaScript program that dynamically generates an HTML table from an array of student objects and appends it to the DOM. (10 Marks)

20. Write a complete form validation script for a student login portal that prevents submission if email or password fields are invalid. (10 Marks)

21. Implement an interactive FAQ accordion component in JavaScript where clicking question headers toggles visibility of answer panels. (10 Marks)

9.4.1 Complete Worked Solutions for Mock Examination 4

Worked Solution: Solutions to Mock Exam 4 (100 Marks)

Section A Solutions (10 × 2 = 20 Marks):

1. Semantic tags improve accessibility for screen readers and boost SEO ranking by clearly defining structural roles. [2M]
2. `<q>` is inline with automatic quotes; `<blockquote>` is an indented block element. [2M]
3. Resets all form control inputs back to their initial default values. [2M]
4. `.content p`. [2M]
5. Capitalizes the first character of each word. [2M]
6. `undefined` indicates a declared variable without value; `null` represents intentional absence of object value. [2M]
7. The scope region before a `let/const` variable is declared where accessing it throws a `ReferenceError`. [2M]

8. Using bracket notation: `obj[variableKey]`. [2M]
9. Adds the CSS class if absent; removes it if already present. [2M]
10. Attaching a single event listener to a parent container to handle events from multiple child elements via event bubbling. [2M]

Section B Solutions (6 × 5 = 30 Marks):

11. `<audio controls> <source src="a.mp3" type="audio/mpeg"> <source src="a.ogg" type="audio/ogg"> </audio>`. [5M]
12. Specificity: Inline style (1000) > ID selector (100) > Class/Pseudo-class (10) > Element selector (1). [5M]
13. Output: "Hello, Kavya" (`this.name` accesses object property). [5M]
14. Palindrome: `function isPal(s) { const c = s.toLowerCase().replace(/[^a-z0-9]/g, ""); return c === c.split("").reverse().join(""); }`. [5M]
15. `querySelectorAll()` returns a static `NodeList` with `.forEach()`; `getElementsByName()` returns a live `HTMLCollection`. [5M]
16. `focus` fires when an element gains active cursor focus; `blur` fires when the element loses focus. [5M]

Section C Solutions (5 × 10 = 50 Marks):

17. **Feedback Portal Form:** Implements form with radio rating group, checkboxes, textarea, file upload, and submit button. [10M]
18. **Product Card Component:** Styled container with `box-sizing: border-box`, `border-radius`, image banner, discount badge, and hover transition. [10M]
19. **Dynamic Student Table Generator:** Iterates array of objects, creates `<table>`, `<tr>`, `<td>` nodes, and appends to container. [10M]
20. **Login Form Validator:** Validates email format with regex, enforces min 8 character password, and alerts errors. [10M]
21. **Accordion FAQ Component:** Attaches click listeners to `.accordion-header`, toggling `.active` class on parent item. [10M]

9.5 University Mock Examination 5

CSE326 Mock Examination 5 (100 Marks — 3 Hours)

Section A: Short Compulsory Questions (10 × 2 = 20 Marks)

1. Explain the function of the `<mark>` tag in HTML5.
2. What is the purpose of `enctype="multipart/form-data"` in HTML forms?

3. How do you specify multiple image formats as fallbacks in HTML5?
4. What is the difference between `padding` and `margin` in the CSS Box Model?
5. Which CSS pseudo-class targets an element when the mouse hovers over it?
6. Explain why `0 == "0"` is true but `0 === "0"` is false.
7. What does `document.createElement()` do?
8. How do you change the text content of a DOM element safely without parsing HTML tags?
9. What is the return value of `document.getElementById()` when no element matches?
10. Explain the role of `event.stopPropagation()`.

Section B: Analytical Medium Problems (6 × 5 = 30 Marks)

11. Construct an HTML5 document boilerplate and explain the purpose of each tag.
12. Write CSS code demonstrating how to create a responsive, centered card container.
13. Predict the output of the following JavaScript code:

```
1 let x = 10;
2 function test() {
3     console.log(x);
4     let x = 20;
5 }
6 test();
7
```

14. Write a function that calculates the sum of all elements in an array using a `for` loop.
15. Explain how `classList.add()` and `classList.remove()` manipulate DOM classes.
16. Differentiate between `click` and `dblclick` events in web applications.

Section C: Comprehensive Programming Problems (5 × 10 = 50 Marks)

17. Design a complete University Course Syllabus page utilizing semantic HTML5 tags, tables, and lists. (10 Marks)
18. Build an interactive timetable table featuring cell merging (`colspan`, `rowspan`) and CSS zebra striping. (10 Marks)
19. Write a complete external CSS stylesheet for an interactive form featuring stylized input focus rings and submit buttons. (10 Marks)
20. Write a complete JavaScript form validation script checking student name, email regex, and password confirmation. (10 Marks)
21. Implement a dynamic real-time search filter in JavaScript that filters table rows as the user types into an input field. (10 Marks)

9.5.1 Complete Worked Solutions for Mock Examination 5

Worked Solution: Solutions to Mock Exam 5 (100 Marks)

Section A Solutions (10 × 2 = 20 Marks):

1. Represents text marked or highlighted for notation/reference (rendered with yellow background). [2M]
2. Required for uploading binary files over HTTP POST without converting payload to text strings. [2M]
3. Using multiple nested `<source>` tags inside `<video>` or `<audio>`. [2M]
4. Padding is transparent space inside the border; Margin is transparent space outside the border. [2M]
5. `:hover`. [2M]
6. `==` coerces string "0" to number 0; `===` compares value and type without coercion (Number vs String). [2M]
7. Instantiates a new unattached HTML element node in memory. [2M]
8. Using `element.textContent = "Text";`. [2M]
9. `null`. [2M]
10. Prevents the event from propagating further up or down the DOM tree (halts bubbling/capturing). [2M]

Section B Solutions (6 × 5 = 30 Marks):

11. Boilerplate: `<!DOCTYPE html>`, `<html lang="en">`, `<head>` with metadata/title, `<body>`. [5M]
12. Card CSS: `.card { width: 300px; margin: 0 auto; padding: 20px; box-sizing: border-box; border-radius: 8px; }`. [5M]
13. **Output:** Throws a `ReferenceError: Cannot access 'x' before initialization` due to the Temporal Dead Zone (TDZ) of block-scoped `let` `x`. [5M]
14. Sum Function: `function sumArray(arr) { let s = 0; for(let i=0; i<arr.length; i++) s += arr[i]; return s; }`. [5M]
15. `classList.add()` inserts a class string into the token list; `classList.remove()` deletes it without affecting other classes. [5M]
16. `click` fires on single button press; `dblclick` fires when two clicks occur within short temporal threshold. [5M]

Section C Solutions (5 × 10 = 50 Marks):

17. **Syllabus Webpage:** Fully semantic document structure with `<header>`, `<main>`, `<section>`, nested lists, and metadata. [10M]

18. **Timetable Table:** Implements `<table>` with `border-collapse: collapse;`, `colspan="5"` for lunch break, `rowspan="2"` for lab, and CSS `tr:hover`. [10M]
19. **Form Stylesheet:** Implements full-width inputs, focus rings via `:focus`, and animated submit button. [10M]
20. **Form Validation Engine:** Intercepts `submit`, validates non-empty name, email regex, password length ≥ 8 , and matching confirmation. [10M]
21. **Real-Time Search Filter:** Attaches `input` listener to search field, loops through `<tr>` elements, checking `row.textContent.toLowerCase().includes(query)` to toggle display. [10M]

9.6 University Mock Examination 6

CSE326 Mock Examination 6 (100 Marks — 3 Hours)

Section A: Short Compulsory Questions ($10 \times 2 = 20$ Marks)

1. What is the purpose of the `<abbr>` tag and its `title` attribute?
2. Explain the difference between `<i>` and `<em>`.
3. What does `<input type="password">` do?
4. Which CSS property controls the spacing between lines of text?
5. What is the specificity value of a class selector?
6. Explain why `typeof []` evaluates to "object".
7. What is the difference between `let` and `const`?
8. How do you select all elements with class "card" using `querySelectorAll()`?
9. What does `element.appendChild()` accomplish?
10. Differentiate between `focus` and `blur` events.

Section B: Analytical Medium Problems ($6 \times 5 = 30$ Marks)

11. Construct an HTML definition list for three computer network protocols.
12. Explain the cascading order of CSS styles when inline, internal, and external styles conflict.
13. Predict the output of the following JavaScript program:

```

1  const car = {
2      brand: "Tesla",
3      getBrand: function () { return this.brand; }
4  };
5  console.log(car.getBrand());
6

```

14. Write a function that counts vowels in an input string.
15. Differentiate between `innerHTML` and `textContent` with security examples.
16. Explain Event Bubbling with an annotated diagram.

Section C: Comprehensive Programming Problems ($5 \times 10 = 50$ Marks)

17. Design a complete University Faculty Profile webpage with semantic tags, profile image, contact details, and qualifications table. (10 Marks)
18. Build an embedded multimedia learning page containing audio and video players with poster frames, fallback formats, and playback controls. (10 Marks)
19. Write a complete external CSS stylesheet for an academic timetable featuring zebra striping, sticky headers, and hover highlights. (10 Marks)
20. Write a complete JavaScript form validation script for a student admission portal checking name, email regex, 10-digit mobile number, and age bounds. (10 Marks)
21. Implement an interactive FAQ accordion component in JavaScript where clicking question headers toggles answer panel visibility. (10 Marks)

9.6.1 Complete Worked Solutions for Mock Examination 6

Worked Solution: Solutions to Mock Exam 6 (100 Marks)

Section A Solutions ($10 \times 2 = 20$ Marks):

1. Defines abbreviations or acronyms; `title` supplies the full expanded term shown on hover. [2M]
2. `<i>` renders italic text without importance; `<em>` applies stress emphasis that alters vocal nuance. [2M]
3. Masks entered characters with dots/asterisks to conceal sensitive passwords. [2M]
4. `line-height`. [2M]
5. Specificity weight of 10 (or (0, 0, 1, 0)). [2M]
6. Arrays are a specialized subtype of objects in JavaScript. [2M]
7. `let` allows variable reassignment; `const` enforces immutable identifier binding. [2M]
8. `document.querySelectorAll(".card")`. [2M]
9. Inserts a node as the last child of the specified parent element. [2M]
10. `focus` triggers when an input gains active cursor; `blur` triggers when it loses focus. [2M]

Section B Solutions ($6 \times 5 = 30$ Marks):

11. `<dl> <dt>HTTP</dt><dd>Web Transfer</dd> <dt>FTP</dt><dd>File Transfer</dd> <dt>SMTP</dt><dd>Mail Transfer</dd> </dl>`. [5M]

12. Priority order: Inline style (1000) > Internal / External style by specificity > Browser default. [5M]
13. Output: "Tesla" (`this.brand` resolves to property on host object). [5M]
14. Vowel counter: `function countVowels(s) { return (s.match(/[aeiou]/gi) || []).length; }`. [5M]
15. `innerHTML` parses tags (vulnerable to XSS); `textContent` inserts safe literal characters. [5M]
16. Event Bubbling propagates events upwards from child target to ancestors. [5M]

Section C Solutions (5 × 10 = 50 Marks):

17. **Faculty Profile Webpage:** Semantic document structure with `<header>`, `<main>`, `<section>`, profile image, and qualifications table. [10M]
18. **Multimedia Webpage:** Constructs `<video controls poster="thumb.jpg"><source src="v.mp4" type="video/mp4"></video>` and `<audio controls>`. [10M]
19. **Timetable Stylesheet:** Implements `border-collapse: collapse;`, `th { background: #0284c7; color: white; },` `tr:nth-child(even) { background: #f8fafc; },` and `tr:hover`. [10M]
20. **Admission Validator:** Validates name, email regex `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`, phone `/^\d{10}$/`, and age `17 ≤ age ≤ 30`. [10M]
21. **Accordion FAQ Component:** Attaches click listeners to `.accordion-header`, toggling `.active` class on parent item. [10M]